

分类号: TP399

单位代码: 10058



天津工业大学
TIANGONG UNIVERSITY

硕士学位论文

论文题目: 基于 Hotstuff 和二叉交流树的
区块链共识机制的设计

学科专业: 控制科学与工程

作者姓名: 陈雨晴

指导教师: 汤春明

完成日期: 2022 年 2 月 26 日

摘要

区块链技术作为一种典型的去中心化技术，被广泛的用来解决权限集中、数据篡改和个人隐私泄露等问题，目前广泛的应用在数字货币、资产管理和选举投票等场景。联盟链作为企业级应用的优先选择而备受关注，针对联盟链中拜占庭容错类的共识机制存在通信复杂度高、视图切换复杂、主节点压力大以及系统规模难以拓展的问题，本文提出了基于 Hotstuff 和二叉交流树的 HSP 共识机制。具体内容如下：

1.针对 Hotstuff 共识机制中 ECDSA(Elliptic Curve Digital Signature Algorithm)算法实现聚合签名和门限签名流程复杂的问题，本文采用了 BLS(Boneh-Lynn-Shacham)数字签名算法。关于门限签名中秘密碎片的分发方式，摒弃了目前使用广泛的 Shamir 秘密分享算法，使用 Feldman 提出的可验证秘密分享算法，解决了秘密持有者作恶和秘密碎片持有者作恶的问题。测试结果表明，在没有视图切换的情况下，在系统总节点数为 4，request/reply=256/256 时，Hotstuff+BLS 较原始 Hotstuff 吞吐量提升了 19.5%，共识延迟降低了 19.9%；当 request/reply=512/512 时，Hotstuff+BLS 较原始 Hotstuff，吞吐量提升了 22.7%，共识延迟降低了 17.3%。

2.针对 Hotstuff 共识机制中，主节点承担的任务量大、主副节点间通信次数多和系统规模难以扩展的问题，本文采用了改进的二叉交流树方法。测试结果表明，在没有视图切换的情况下，当系统总节点数为 64，request/reply=256/256 时，HSP 较 Hotstuff+BLS 吞吐量提升了 33.8%，共识延迟下降了 16.4%；当 request/reply=512/512 时，HSP 较 Hotstuff+BLS 吞吐量提升了 69.0%，共识延迟下降了 11.2%。在节点总数为 64，request/reply=256/256 时，在由于网络故障而发生视图切换的情况下，HSP 较 Hotstuff 吞吐量提升了 35.1%，共识延迟下降了 22.6%；在主节点连续故障而连续进行视图切换的情况下，HSP 较 Hotstuff+BLS 吞吐量提升了 34.4%，共识延迟降低了 17.9%。

关键字：共识机制；门限签名；二叉交流树

Abstract

As a typical decentralized technology, blockchain technology is widely used to solve the problems of authority concentration, data tampering and personal privacy leakage. At present, it is widely used in digital currency, asset management, election voting and other scenarios. Consortium chain, as the preferred choice for enterprise-level application implementation, has attracted much attention. For the Byzantine Fault Tolerant consensus in the consortium blockchain, there are problems such as high communication complexity, complicated view changing, high pressure on the primary, and difficulty in expanding the system scale. The HSP consensus based on binomial swap forest and Hotstuff is proposed. The details are as follows:

1. Aiming at the complexity of the Elliptic Curve Digital Signature Algorithm in the Hotstuff consensus mechanism, the realization of the aggregation signature and threshold signature process is complex, this paper uses the BLS digital signature algorithm. Regarding the distribution method of the secret fragments in the threshold signature, the widely used Shamir secret sharing algorithm is abandoned, and the verifiable secret sharing algorithm proposed by Feldman is used to solve the evil of the master secret holder and the evil of the secret fragment holder. The test results show that, without view changing, when the total number of nodes in the system is 4 and request/reply=256/256, Hotstuff+BLS increases the throughput by 19.5% compared with the original Hotstuff, and reduces the consensus delay by 19.9%; When request/reply=512/512, compared with the original Hotstuff, the throughput of Hotstuff+BLS is increased by 22.7%, and the consensus delay is reduced by 17.3%.

2. Aiming at the problems of the large amount of tasks undertaken by the master node, the number of communications between the master and the backups, and the difficulty of system expansion in the Hotstuff consensus mechanism, this paper adopts an improved binomial swap forest method. The test results show that without view changing, when the total number of nodes in the system is 64 and request/reply=256/256, HSP increases the throughput of Hotstuff+BLS by 33.8%, and the consensus delay decreases by 16.4%; when request /reply=512/512, the throughput of HSP is 69.0% higher than that of Hotstuff+BLS, and the consensus

delay is reduced by 11.2%. When the total number of nodes is 64 and request/reply=256/256, in the case of view changing due to network failures, HSP has increased the throughput of Hotstuff by 35.1%, and the consensus delay has decreased by 22.6%; when the master node fails continuously, in the case of continuous view switching, compared with Hotstuff+BLS, the throughput of HSP is increased by 34.4%, and the consensus delay is reduced by 17.9%.

Keywords:consensus mechanism; threshold signature; binomial swap forest

目 录

第一章 绪论.....	1
1.1 课题研究背景与意义.....	1
1.2 国内外研究现状.....	2
1.2.1 区块链研究现状.....	2
1.2.2 共识机制研究现状.....	3
1.3 课题研究目的.....	5
1.4 课题研究内容.....	6
1.5 本论文结构安排.....	7
第二章 区块链和共识机制的理论基础	9
2.1 区块组成.....	9
2.2 数字签名.....	10
2.2.1 ECDSA.....	10
2.2.2 Schnorr	11
2.2.3 BLS	12
2.3 秘密分享.....	15
2.3.1 Shamir 秘密分享	15
2.3.2 Feldman 秘密分享.....	15
2.4 数据聚合.....	16
2.4.1 中心化聚合	17
2.4.2 二叉交流树.....	17
2.5 共识机制.....	18
2.5.1 PBFT	18
2.5.2 SBFT	20
2.5.3 Hotstuff.....	21
2.6 本章小结.....	23
第三章 HSP 共识机制设计	25
3.1 BLS 门限签名	25
3.1.1 公私钥初始化.....	25
3.1.2 消息签名.....	26
3.1.3 聚合签名及验证.....	26
3.2 改进二叉交流树.....	27
3.2.1 不存在恶意节点的信息交换方式.....	27
3.2.2 分根节点为恶意节点的信息交换方式	28
3.3 HSP 共识流程	29

3.3.1 主节点选择.....	30
3.3.2 HSP 正常共识流程	30
3.3.3 HSP 视图切换流程	31
3.4 安全性分析.....	32
3.5 本章小结.....	33
第四章 HSP 共识机制测试及结果分析	35
4.1 实验环境.....	35
4.2 实验设计	35
4.3 实验结果展示及分析.....	36
4.3.1 数字签名实验结果及分析.....	36
4.3.2 二叉交流树实验结果及分析.....	37
4.3.3 视图切换实验结果及分析.....	38
4.4 本章小结.....	40
第五章 总结与展望	41
5.1 总结	41
5.2 展望.....	42
参考文献.....	43

第一章 绪论

1.1 课题研究背景与意义

习近平总书记在 2019 年强调,要把区块链作为核心技术自主创新的重要突破口,加快推动区块链技术和产业创新发展。自此区块链技术在我国逐步走入大众视野,使得非科研人员对区块链技术的兴趣浓烈,科研人员纷纷加入到区块链技术的研究队伍。截止到 2020 年底,我国各省市级单位发布与区块链技术相关的文件共 227 份^[1],无不显示出我国积极发展区块链产业的决心。

区块链^[2]的概念于 2008 年由化名为“中本聪”的学者首次提出,具体根据应用场景的不同,分为公有链、私有链和联盟链^[3]。公有链,即未对节点设置准入门限,所有节点均可在注册成功后随时加入区块链网络并在该网络中进行交易。联盟链,对节点设置了准入门限,即只有通过身份验证的节点才可加入系统,主要服务于各类机构组织。私有链,一般要求系统内节点全部诚实、可信,大多应用于企业内部,适用场景少。区块链技术发展至今,经历了三个阶段,最早期的区块链的典型代表为比特币,其类型为公有链。比特币是目前为止最为成功的区块链技术应用,将去中心化特性和金融场景结合,为后期的加密货币提供了原型,为公有链系统开发提供了基础思路。随着业务场景复杂化,区块链技术也随之发展,进入到以以太坊(Ethereum)为代表的时代,以太坊也是公有链类型的应用,引入了智能合约的概念,为区块链技术扩展了除金融领域外的其他适用场景,但主要应用场景依旧为金融领域。考虑到监管需求和吞吐量等性能需求,区块链相关技术人员逐步将关注点由公有链转移到联盟链,目前,区块链技术发展到以超级账本项目(Hyperledger)为代表的时代。Hyperledger 是联盟链应用的典型代表,为区块链技术在企业级应用提供了平台,其成员认证、可插拔式共识协议和链码等服务使其能适用于不同的业务场景。

区块链作为一种典型的分布式系统,具有去中心化的特点。区块链的去中心化特性与监管存在矛盾:在公有链系统中,由于缺乏监管,容易招致各种类型的攻击,如分叉攻击、女巫攻击和重入攻击等。任何一种成功攻击都会对链造成难以估计的损失,所以缺乏监管制约了其在金融等领域的发展。由于金融等领域需要在可控环境中部署区块链技术,联盟链的准入机制很好满足了可控性需求。目前,联盟链在供应链金融、医疗、教育、数字版权、智能家居和智慧城市等方面均有良好的应用前景。联盟链作为企业应用落地的首要选择,具有重要研究意义。

区块链的去中心化特性需要共识机制来使系统的状态达成一致。随着科技水

平的提高和网络的普及,每天产生巨大的交易量,如果交易需要在区块链中完成,保证交易即时正确执行是共识机制需要解决的问题。目前的共识机制无法满足高吞吐量、低延时的要求,并存在浪费资源、去中心化程度低以及扩展性较差等问题。因此,国内外很多科研人员投身于共识机制优化算法^[4-7]的研究中。

针对公有链的工作量证明(Proof of Work, PoW),其改进算法有权益证明^[8](Proof of Stake, PoS)、委托权益证明^[9](Delegate Proof of Stake, DPoS)和 GHOST^[10](Greedy Heaviest Observed Sub-Tree, GHOST)等。针对私有链的 Paxos 共识机制^[11-12],其改进算法有 Cheap-Paxos^[13]、MultiPaxos^[14]和 Raft^[15]等。联盟链典型共识机制为实用拜占庭容错算法^[16](Practical Byzantine Fault Tolerance, PBFT),其改进算法有动态拜占庭容错算法^[17]、主动动态授权拜占庭容错算法^[18]、可扩展的拜占庭容错算法^[19](Scalable Byzantine Fault Tolerance, SBFT)和 Hotstuff^[20-21]等,以上共识机制针对 PBFT 存在的问题均进行了不同方向的改进,但均未解决 PBFT 存在的所有问题,不具有普适性。因此研究适用于联盟链的、吞吐量高、共识延迟低、扩展性强、视图切换简单的共识机制具有重要意义。

1.2 国内外研究现状

1.2.1 区块链研究现状

区块链技术起源于 2008 年,2009 年,比特币支付系统正式诞生,比特币是区块链技术在金融方面的一个典型应用且是迄今为止最为成功的区块链应用场景,是区块链 1.0 时代的典型代表,此时关于区块链的研究局限于金融方面。区块链作为加密货币的底层技术,区块链发展的同时,加密货币^[22]也发展迅速,目前有 800 种以上的加密货币在流通,市场份额占比比较大的货币有比特币、以太坊、币安币、DOT 币、瑞波币和艾达币等。上述加密货币均没有国家主权背书,是“不稳定”的。2019 年第一版 Libra 白皮书正式发布,它是一种相对“稳定”的加密货币,最初由法币计价的低波动性的资产背书,它的出现再一次激发了人们对稳定币的热情。与此同时,不少国家已开展央行数字货币的发行试验工作,央行数字货币是由国家主权信用背书,具有法定地位。e-Dinar 是第一个国家数字货币,中国早在 2014 年就开始推动央行数字货币的发行。

2013 年以太坊白皮书^[23]正式发布,该文件中提出以区块链技术为底层支撑去构建去中心化应用的思路。2015 年 7 月,以太坊正式提供数字货币支付服务与智能合约服务,标志着区块链进入了 2.0 时代。以太坊的开发参考了比特币系统,其最大的创新就是引入了智能合约的概念。可以简要的理解为比特币是黄金,其价值在于本身,而以太坊是一个平台,其价值是为各类应用程序提供技术支持。智能合约服务使得区块链业务场景跳离金融领域成为了可能,解决了比特币系统

存在的应用场景单一、扩展性不足的问题。但是，区块链 2.0 仍主要应用于金融领域。

2015 年 12 月，Linux 基金会宣布了 Hyperledger 项目的启动，目前超级账本项目有十余个子项目，如 Fabric、Sawtooth、Burrow 和 URSA 等，涉及到生活中的各个领域。Hyperledger 的出现标志着区块链技术迈入了区块链 3.0 时代。区块链 3.0 时代的应用场景完全跳出了金融类，涉及到物联网、供应链和政务等各个领域。Fujimura 等人提出了去中心化权力系统^[24]的概念，赵庭悦指出了区块链技术在电商供应链物流管理方面^[25]的应用，陈锋平等人阐述了区块链技术在图书馆书籍借还方面^[26]的应用，Dorri 等人以智能家居为例介绍了区块链在物联网隐私和安全方面的应用^[27]，金凯和周秀秀研究了区块链在食品供应链溯源方向的应用^[28-29]，张国潮等人研究了区块链在数字音乐版权管理方面^[30]的应用，徐龙等人研究了区块链在电力物联网方面^[31]的应用，杨德昌、赵肖余等人分析了区块链技术在能源互联网中的应用^[32]现状并提出了展望。

目前，国外主要的区块链平台有以太坊和 Hyperleger，国内具有代表性的区块链平台是分布式应用账本开源社区，重点孵化的开源项目有众安链(Annchain)和 BCOS(Be Credible, Open & Secure, BCOS)。Annchain 是国内首个基于有向无环图架构且支持智能合约的区块链平台，目前有多个应用落地，如自动理赔和电子保单存证解决方案和步步鸡区块链养殖项目等，覆盖金融、农业和物联网等众多领域。BCOS 在设计上更贴近商业需求，能够提供监管和支持审计的特性，使其在金融领域具有良好的应用前景，同时共识机制的插件化使其可以满足于不同的业务场景需求。近年，基于 BCOS 的微粒贷备付金管理及对账平台、供应链金融服务平台和股权登记与服务平台等相继落地。截止到 2021 年二季度，BCOS 社区开发者成员已达数千，数百个应用落地。

1.2.2 共识机制研究现状

针对不同的应用场景，需要不同类型的区块链，不同类型的区块链对应不同的共识机制^[33]。

在公有链中，最广为人知的共识机制便是 PoW，中本聪在 2008 年采用了这种机制，也称中本共识。中本共识中涉及的时间戳技术^[34-36]和默克尔树技术^[37-39]是由 Stuart Haber 以及 Scott Stornetta 提出；点对点(Peer To Peer, P2P)交易以及分布式存储的概念由戴伟于 1998 年在 B-money 中提出。中本共识中最核心的设计思想来源于垃圾邮件的解决方案^[40]，即在发送邮件消息之前，要求用户发送与邮件消息相关的工作证明，这种证明通常是解决一个数学难题。中本共识采用的数学难题是将一个 256 位的整数 Nonce 加入到区块头中，确保区块整体的哈希值小于一个固定的值。首先发现 Nonce 的矿工获得记账权和记账奖励，寻找 Nonce

的过程就是一个不断试数的过程, 因此算力高的计算机寻找到 Nonce 的几率大。

中本共识的缺点是造成大量能源浪费, 同时由于矿池造成的算力集中, 存在 51%攻击的可能性, 而且此种共识机制的吞吐量低, 平均为 7TPS(Transaction Per Second, TPS)。针对中本共识存在的问题, 目前出现了多种改进方案, 如将在寻找合适 Nonce 时进行的无意义的哈希运算转换成寻找大素数的运算。针对采用 ASIC (Application Specific Integrated Circuit) 矿机进行挖矿而导致的算力集中的问题, Tromp 等人提出了一种基于内存消耗的反 ASIC 矿机算法^[41]。针对消耗能源和存在的 51%攻击的问题, 2011 年 7 月, Quantum Mechanic 首次提出了 PoS 共识机制, 并于 2012 年在点点币中首次实现, PoS 引入“币龄”的概念, 拥有币龄高的节点获得记账权的可能性大, 但 PoS 也存在缺陷, 首先是初始货币的分配问题, 其次是鼓励囤积行为, 所谓囤积行为即节点堆积“币龄”, 堆积币龄时节点离线, 只有在“币龄”积攒到一定数量后才会上线, 此种行为容易招致网络攻击。此外, PoS 最大问题的是会招致无差别攻击^[42]和远程攻击^[43]。为了进一步加快交易速度, 解决 PoS 中离线节点的积累币龄的安全问题。Daniel Larimer 于 2014 年提出了 DPoS 共识机制, 引入了代表(delegate)和见证者(witness)的概念, 它们均由系统内节点投票选出。delegate 按指定顺序依次把交易打包成区块, 其余 delegate 负责对区块投票; witness 只需要跟随其 delegate 的投票即可。这样可大大缩减共识延迟, 同时吞吐量可达到每秒数千笔。但 DPoS 也存在缺陷, 如 DPoS 较中本共识和 PoS 去中心化程度较弱。针对中本共识还有一种改进方向为有向无环图(Directed Acyclic Graph, DAG), 其中比较典型的有 GHOST。中本共识的成链规则为最长链即为有效链, GHOST 的成链规则不同于中本共识, 最重链即为有效链, 允许链出现分叉并且承认一部分分叉。目前比较典型的 DAG 类共识机制有 Casper^[44]、Bitcoin-ng^[45]、Phantom^[46]和 SPECTRE^[47]等, 如何高效解决双花问题^[48]是 DAG 类共识机制面临的问题。

在私有链中, 比较典型的共识机制有 Paxos^[12]和 Raft^[15]。Paxos 算法最早应用在分布式系统中, Paxos 需要两次信息交互才可确认提案; 针对此问题, Multi-Paxos^[14]做出了改进, Multi-Paxos 是对连续多个值达成一致, 但存在“续租”问题。为解决“续租”问题, Raft 算法于 2014 年被 Ongaro 等人提出。Raft 算法需要领导(Leader), Leader 可随时向其他副本节点发送心跳来维持自己的统治, 不存在续租问题, Raft 存在的问题是适用范围小且只能应用在完全可信环境。

在联盟链中, 较为典型的共识机制是 PBFT 共识机制。PBFT 由拜占庭将军问题^[49]演化而来。1999 年, Miguel Castro 与 Barbara Liskov 提出了 PBFT 算法, 该算法使得系统可以每秒处理上千的请求, PBFT 将系统内节点分为主节点和副本节点。主节点的工作有接收请求、验证请求、打包区块和发布区块; 副本节点的

工作仅有验证区块。主节点为非拜占庭节点时，PBFT 达成共识需要两次全网通信，通信复杂度为 $O(n^2)$ ；主节点为拜占庭节点时，通信复杂度为 $O(n^3)$ ，其中 n 为系统内节点总数。可见，随着系统内节点数的增多，系统的通信复杂度大幅度提高。孙一蓬于 2019 年提出的 DDBFT 算法^[17]将系统内部节点按 3:2 的比例划分为共识节点和非共识节点，仅共识节点参与共识过程。此种行为提升了共识速度，但降低了系统容错率，同时由于非共识节点没有工作量，系统存在节点工作量分配不均匀的问题，且系统的通信复杂度依旧是 $O(n^2)$ 级别。王海勇等人于 2019 年提出的 IDBFT 算法^[18]解决了工作量分配不均的问题，该算法将系统内部节点按 1:1 的比例划分为共识节点和普通节点，共识节点和普通节点的角色可进行切换，在共识节点全部诚实的情况下就执行简化共识流程；若共识节点中有一个拜占庭节点，则共识节点执行类似 PBFT 的两阶段共识流程。由于存在共识流程切换的步骤，共识过程会消耗多余时间。同时，由于参与共识的节点数量减少，系统的容错率也降低了，且系统的通信复杂度依旧是 $O(n^2)$ 级别。Zyzyva 算法^[50]于 2007 年被 Ramakrishna Kotla 等人提出，该算法使用了收集器技术降低了通信复杂度，整个共识过程中不存在消息全网广播，通信复杂度为 $O(n)$ 。但 Zyzyva 也存在一个致命缺点，即不支持轻量级客户端，客户端在请求执行的过程中必须保证在线，否则不能完成共识。为解决此问题，Gueta 和 Abraham 等人于 2019 提出了 SBFT 共识机制^[19]。SBFT 依旧将系统内节点划分为主节点和副本节点。和 Zyzyva 不同，副本节点将预执行的结果返回至主节点，主节点充当收集器的角色，解决了 Zyzyva 要求客户端长期在线的问题。当主节点为非拜占庭节点时，通信复杂度为 $O(n)$ ；主节点为拜占庭节点时，通信复杂度为 $O(n^2)$ 。可见在进行视图切换时，系统的通信复杂度依旧为平方级别；为降低系统在进行视图切换时的通信复杂度，尹茂帆等人于 2019 年提出了 Hotstuff 共识机制^[20-21]，将传统的两阶段共识转换为三阶段共识，将视图切换流程融入正常的共识流程，通信复杂度为 $O(n)$ 。Hotstuff 存在实现聚合签名和门限签名效率低下和主副节点工作量分配不均衡的问题。

1.3 课题研究目的

随着互联网的快速发展，生活中每天产生的交易量是无比巨大的，如何保证交易安全快速的正确完成以及交易相关数据的安全传输及存储是目前面临的问题。传统的交易执行方式是中心化的，即用户在中心化平台注册账户，获得由平台给予的账户地址，用户向该账户地址里充值数字资产，便可在此平台进行交易，平台为用户提供服务，交易成功后，用户账户中的资产余额发生变化。中心化交易方式存在以下三个主要问题：第一，资产安全风险，用户只是拥有在该平台的

账户密码，并不能实际掌控自己的资产，一旦平台的钱包受到攻击或者平台自身作恶，用户资产将受到威胁；第二，资产控制限制，平台会设置规则限制用户的提现，包括限制提现额度、提现时间、设置提现手续费等，影响用户对自己资产的自由掌控；第三，交易清算不透明，交易清算过程由平台完成，无法追溯，交易所可轻易制造虚假交易。由于中心化交易方式存在以上问题，去中心化交易方式应运而生。区块链发明之初，就是为了解决“中心化”带来的问题。

区块链作为典型的分布式系统，需要共识机制来保证系统内节点状态的一致性，进而保证交易被正确有序执行，因此共识机制性能的好坏直接影响到整个系统的性能。一个性能良好的共识机制可以提升系统的吞吐量、降低系统共识延迟并且使得系统具有良好的可扩展性。在实际生活中，联盟链作为应用落地的首要选择，针对联盟链中拜占庭容错类算法，国内外的研究人员进行了多项研究，提出了多种共识机制，但是都存在着一些问题，比如难以拓展、通信复杂度高、视图切换复杂、主节点压力大等。因此，本文研究目的是设计出一种易拓展、通信复杂度低、视图切换简单且节点压力平均的共识机制。

1.4 课题研究内容

本文选取联盟链的共识机制作为研究对象，具体研究内容如下：

(1) 针对拜占庭容错类算法中，主副节点间通信复杂度高的问题，采用了收集器技术和 BLS 数字签名算法。首先将系统内节点划分为主节点和副本节点，主节点接收来自客户端的交易、对交易进行验证，通过验证后为交易分配全局唯一的序列号。主节点收集到一定数量的交易后，将交易打包成区块并将该区块广播至全部副本节点，副本节点接收到区块后，对区块内容进行投票，返回投票信息给主节点。投票信息其实就是各副本节点针对此区块的签名，主节点收到一定数量的签名，对签名进行聚合，重复此行为三次，最后一次副本节点发送消息至客户端，表明此副本节点已经执行此请求。

(2) 针对拜占庭容错类算法中，主节点处理任务多、压力大的问题，本文采用了改进的二叉交流树技术。改进的二叉交流树技术分散了签名聚合的压力，将主节点的压力分散至各分主节点，在一定程度上提升了区块的验证速度，同时改进的二叉交流树技术中的主节点每次只与 $\log_2^n$ 个副本节点通信，在节点数量较多的情况下，极大的减少了主副节点间的交易次数。

(3) 针对拜占庭容错类算法中，视图切换复杂的问题，采用了三阶段确认方式。PBFT 为两阶段确认方式，此种共识方式在主节点故障的情况下进行视图切换的通信复杂度为 $O(n^3)$ 。HSP 共识机制增加一轮确认，可将视图切换的通信复杂度降到 $O(n)$ 。一阶段确认只能在没有视图切换的情况下保证交易的活性和

一致性，因此考虑两阶段确认。两阶段确认在出现视图切换时依旧可以保证交易的活性和一致性，但两阶段确认的视图切换过程通信复杂度过高。Tendermint 引入了加锁解锁机制，简化了视图切换的过程。但 Tendermint 中加锁解锁机制会导致活性的丢失，于是 Hotstuff 又增加了一阶段确认。第三阶段确认主要是为了解决加锁解锁机制造成的活性损失。本文参考 Hotstuff 共识机制，决定采用三阶段确认方式，在保证系统活性和一致性的同时，降低视图切换时的消息复杂度。

1.5 本论文结构安排

本文的具体结构安排如下：

第一章：绪论。首先概述了本课题的研究背景及意义，然后分别介绍了区块链和共识机制的国内外研究现状，通过分析研究背景和前人研究现状介绍了研究本课题的目的，接着介绍了本课题研究的内容，最后概述本论文的结构安排。

第二章：区块链和共识机制的理论基础。首先介绍了区块的组成。然后介绍了用来保证区块链的匿名性、可验证性和不可篡改性的数字签名算法，详细介绍了目前使用广泛的 ECDSA、Schnorr 和 BLS 签名算法的签名过程和验证过程。同时还介绍了三类算法实现聚合签名和门限签名的具体过程，并分析了各算法的缺点。接着介绍了门限签名实现涉及的秘密分享方式，详细的介绍了 Shamir 秘密分享方案和 Feldman 提出的可验证秘密分享方案。为了实现密钥聚合，接着详细的介绍了中心化聚合和二叉交流树聚合两种数据聚合方式并进行了优缺点分析。最后介绍了目前较为典型的联盟链共识机制，包括 PBFT、SBFT 和 Hotstuff，分别介绍三种共识机制的共识流程，并分析了三种共识机制的优缺点。

第三章：HSP 共识机制设计。本章首先介绍了 BLS 门限签名算法的实现，详细的阐述了公私钥初始化、消息签名和聚合签名的生成和验证过程。然后分析了传统的二叉交流树的缺点，介绍了改进的二叉交流树技术，并详细阐述了算法流程，同时考虑到了分根节点为故障节点的情况并给出了相应的解决方式。紧接着，在介绍 HSP 共识机制的共识流程时，首先介绍了主节点选择的方式。然后考虑到系统可能由于节点故障或网络延迟等原因出现视图切换的情况，分两种情况阐述了共识流程。最后，从一致性和活性两个方面分析了 HSP 共识机制的安全性。

第四章：HSP 共识机制测试及结果分析。首先介绍了实验环境，然后根据两个改进方向，设计了三组实验。第一组实验用来测试 BLS 签名算法对共识机制性能的影响；第二组实验用来测试在无拜占庭节点的情况下改进的二叉交流树对共识机制性能的影响；第三组实验用来测试在系统中出现视图切换的情况下改进的二叉交流树对系统性能的影响。然后分别对实验结果进行了展示，最后对实验

结果进行了分析。

第五章：总结与展望。对本文工作进行总结，对 HSP 共识机制的创新点和实验结果进行总结。最后总结了课题存在的问题并对未来的研究方向提出了展望。

第二章 区块链和共识机制的理论基础

2.1 区块组成

区块链由一个个区块相互链接而成，单个区块的数据结构如图 2-1 所示。区块链中的每个区块均由区块头和区块体两部分组成，其中区块头是整个区块的核心。以比特币系统为例，区块头的数据可以分为三部分。第一部分是父块哈希值，此数据用于将此区块与上一区块链接。区块的哈希值由区块头通过哈希计算得到，不涉及区块体。不考虑哈希碰撞^[52]的情况下，区块哈希值可以唯一的标识一个区块。如图 2-1 所示，BlockB 的头部信息中需要包含 BlockA 的哈希值，此种行为提高了区块的篡改难度，从而提高链的可靠性^[53-54]。假设目前系统中存在一条链，区块的顺序如图 2-1 所示为 BlockA-BlockB-BlockC，若此时攻击者想要篡改 BlockA 中的数据，则该攻击者不仅需要修改 BlockA 的数据，还要修改 BlockB 和 BlockC 的数据，此种行为会极大的增加攻击成本，因此区块的链式结构可以保证越早的块越安全。

第二部分包括挖矿难度（Difficulty）、时间戳（timestamp）和随机数（Nonce）。Difficulty 代表出块难度，时间戳代表此区块产生时间，Nonce 为矿工计算出的一个数字，是挖矿成功的证明。

第三部分是默克尔（Merkle）根。Merkle 根是由 Merkle 树计算得来。

区块体用来存储交易信息，该信息以 Merkle 树形式存储。Merkle 树是一种哈希二叉树，用来表示区块中的所有交易。

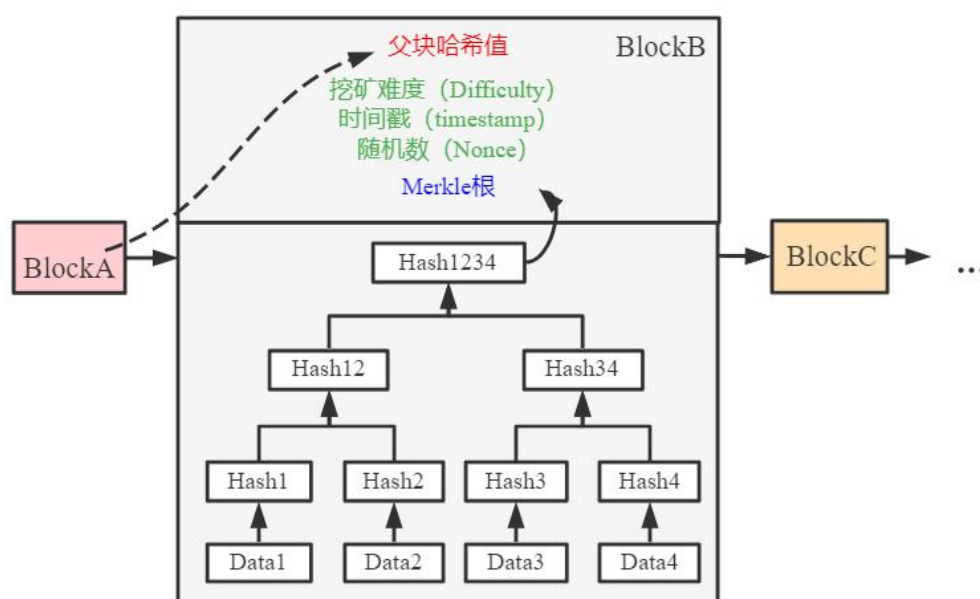


图 2-1 区块结构

2.2 数字签名

区块链中使用数字签名^[55]技术来保证交易的可验证性和完整性。为提升区块验证速度，门限签名技术和聚合签名技术被应用到区块链中。门限签名技术是指根据系统需求将秘密分成若干份秘密碎片，将秘密碎片分发给系统内节点，只有聚集起满足阈值数量的秘密碎片才可以恢复出最初的秘密。聚合签名技术将若干个签名通过简单运算合成一个签名。不同的数字签名算法实现门限签名和聚合签名的难度是不同的。

常用的数字签名算法有 RSA^[56-57]、椭圆曲线签名算法^[58-59] (Elliptic curve Digital Signature Algorithm, ECDSA)、Schnorr^[60] 和 BLS^[61-62] 等。按照其先进性本文选取 ECDSA、Schnorr 以及 BLS 进行了介绍，并分别介绍了上述三类算法的聚合签名^[63]实现和门限签名实现。

2.2.1 ECDSA

ECDSA 使用十分广泛，如最初版本的比特币便使用 ECDSA 来生成账户的公私钥以及对交易和区块进行验证。

ECDSA 的签名过程可分为五步。表 2-1 为具体的参数说明。

(1) 计算消息 msg 的哈希值 e ，计算方法如式(2-1)，选择 e 的前 L_n 位记作 z ，其中 L_n 是 n 的比特长度， z 在数值方面可以大于 n 但长度不能长于 n ， n 是任意大整数；

$$e = HASH(msg) \quad (2-1)$$

(2) 随机选取一个整数 k ， k 的取值范围为 $[1, n-1]$ ，计算点 (x_1, y_1) 的坐标。计算方法如式(2-2)：

$$(x_1, y_1) = k \times G \quad (2-2)$$

(3) 计算第一部分签名 r ，计算方法如式(2-3)，如果 $r=0$ ，则返回第二步：

$$r = x_1 \bmod n \quad (2-3)$$

(4) 计算第二部分签名 s ，计算方法如式(2-4)，如果 $s=0$ ，则返回第二步：

$$s = k^{-1}(z + rd_A) \bmod n \quad (2-4)$$

(5) 最终的签名为 (r, s) 。

在签名的过程中，要求整数 k 是私密的。针对不同的消息进行签名时需选取不同的整数 k ，否则，存在密钥泄露的风险。为确保 k 对于每条消息都是唯一的，可不采用随机数生成的方式，也可通过一些数学运算来得出与消息和私钥有所关联的 k ，可以理解为同一个用户对不同的消息进行签名时的 k 是不同的，不同的用户对同一个消息进行签名时的 k 是不同的。

ECDSA 签名验证过程可分为五步：

(1) 验证公钥 Q_A 是否等于 O ，并且验证其坐标是否有效，检查 Q_A 是否在曲线上，查检查 $n \times Q_A = O$ ；

(2) 验证 r, s 是否为位于 $[1, n-1]$ 之间的整数，如果不是，则签名无效；

(3) 计算 $e = \text{HASH}(msg)$ ，选择 e 的前 L_n 位记作 z ；

(4) 计算点 (x_1, y_1) ，涉及的计算公式如(2-5),(2-6)和(2-7)，如果 $(x_1, y_1) = O$ ，则签名是有效的：

$$u_1 = zs^{-1} \bmod n \quad (2-5)$$

$$u_2 = rs^{-1} \bmod n \quad (2-6)$$

$$(x_1, y_1) = u_1 \times G + u_2 \times Q_A \quad (2-7)$$

(5) 最后，如果等式 $r \equiv x_1 \pmod{n}$ 成立，则签名是有效的；否则，无效。

表 2-1 ECDSA 参数

参数	概述
$Ep(a, b)$	椭圆曲线参数
p	模运算的底
G	基点
n	n 是 G 的素数整数阶，意味着 $n \times G = O, O$ 是标识元素
d_A	私钥（随机选择）
Q_A	公钥（由私钥计算得到： $Q_A = d_A \times G$ ）
msg	待签名的消息

由式(2-4)可知，随机数 k 作为乘数出现在签名中，即 ECDSA 算法生成的签名对于随机数 k 来说是非线性的。如果将由 ECDSA 生成的数字签名进行简单加减运算而得到聚合签名，该聚合签名在验证的时候较为复杂，若想要简化验证过程，则需要对生成聚合签名的流程进行复杂处理。可见，ECDSA 实现聚合签名较为复杂，因此不做过多介绍。ECDSA 可用来实现门限签名，但实现流程较为复杂，此处不予介绍。

2.2.2 Schnorr

2018 年 7 月，比特币开发者 Pieter Wuille 提出了将比特币的签名算法更改为 Schnorr 签名算法。在 Bitcoin Core 0.21.0 版本中，数字签名算法已升级为 Schnorr。Schnorr 签名计算过程很简单，其中具体参数意义如表 2-2 所列，具体步骤如下：

(1) 选择随机数 k ，计算点 $R = k * G$ ；

(2) 计算签名 s ，计算方法如式(2-8)所示：

$$s = k + H(R \parallel P \parallel msg) * pk \quad (2-8)$$

(3) 向其它节点广播签名及相关信息。各节点验证 Schnorr 签名的正确性, 验证方式如式(2-9)所示, 如果等式成立, 表明签名通过验证:

$$sG = R + H(R \parallel P \parallel msg) * pk \quad (2-9)$$

表 2-2 Schnorr 签名参数

参数	简述
G	群的生成元
pk	私钥
P	公钥, 且 $P = pk * G$
msg	待签名消息

Schnorr 签名算法可实现聚合签名, 以两个参与方聚合签名为例: 参与方 u_1 和 u_2 的公钥为 P_1 和 P_2 , 私钥为 x_1 和 x_2 , k_1 和 k_2 为随机数, $P = P_1 + P_2$ 为组公钥。 u_1 的签名为 (R_1, s_1) , u_2 的签名为 (R_2, s_2) , 两个签名相加得到组签名 (R, s) 。其中: $R = R_1 + R_2, s = s_1 + s_2$ 。

由式(2-8)可知, 随机数 k 作为加数出现在 Schnorr 签名中, 因此在聚合签名的时候, 将多个签名进行简单组装即可, 实现聚合签名较为容易, 验证聚合签名时也较为容易。但进行签名聚合时, 参与方需要先相互交换公钥和 R 值, 然后再进行各自的签名, 最后进行聚合, 节点间需要多轮通信, 同时要求节点保持在线。

Schnorr 算法也可实现门限签名。(a,b)门限签名的意思为: 假设系统内共有 b 个签名者, 只有当收集到的签名数量大于等于 a 时, 才能将此 a 份签名聚合成为完整签名, 其中 $a < b$ 。目前采用 Schnorr 算法来实现 (p,p) 门限签名^[63]较多, 即所有参与方都贡献自己的签名后才可以聚合成完整签名。Schnorr 签名算法也可用来实现 (p,q) 门限签名, 其中 $p < q$ 。实现 (p,q) 门限签名需要构造一棵全部公钥的默克尔树, 该默克尔树在节点数量较多的情况下会变得十分庞大, 使得签名效率低下。由于实现 (p,q) 门限签名较为复杂, 所以一般实现 (p,q) 门限签名时, 不考虑使用 Schnorr 算法。

Schnorr 的缺点有: 1) 聚合签名需要多次通信, 需要节点长期保持在线; 2) 聚合签名算法依赖随机数生成器; 3) 实现 (p,q) 门限签名需要构建公钥的默克尔树, 当 p 和 q 较大时, 树所占空间会相当大。

2.2.3 BLS

BLS 数字签名算法由 Dan Boneh 等人于 2001 年提出^[61]并且于 2018 年对该算

法进行了更新^[62]。BLS 采用了基于双线性映射的椭圆曲线配对技术^[64-66]，来实现签名验证与聚合。

BLS 算法的签名过程比较简单，可分为准备、签名和验证三个阶段。BLS 聚合签名的参数说明如表 2-3 所列。

(1) 准备阶段：秘密选取随机数字作为私钥 sk ，计算公钥 pk ，公钥的计算公式如式(2-10)：

$$pk = g^{sk} \quad (2-10)$$

(2) 签名阶段：待签名的消息为 msg ，首先将消息 msg 的哈希 $H(msg)$ 映射到曲线上，然后计算签名，签名的计算方法如式(2-11)：

$$\sigma = H(msg)^{sk} \quad (2-11)$$

(3) 验证阶段：验证签名是否正确即为验证双线性映射是否配对成功，即为验证等式(2-12)是否成立：

$$e(\sigma, g) = e(H(msg), pk) \quad (2-12)$$

BLS 签名算法可实现聚合签名。给定公钥 pk_i 、需要签名的消息 msg_i 和签名 σ_i ，即可验证聚合签名的正确性。聚合签名的计算公式如式(2-13)：

$$\sigma = \sigma_1 \sigma_2 \dots \sigma_n \quad (2-13)$$

关于聚合签名的验证部分，根据签名信息的不同，验证方式是不同的。若 n 个用户的签名是针对同一个消息 msg 的，则验证签名即为验证式(2-14)是否成立：

$$e(\sigma, g) = e(H(msg), pk_1 pk_2 \dots pk_n) \quad (2-14)$$

若 n 个用户的签名是针对不同消息的，验证过程为验证式(2-15)是否成立：

$$e(\sigma, g) = e(H(msg_1), pk_1) e(H(msg_2), pk_2) \dots e(H(msg_n), pk_n) \quad (2-15)$$

上述聚合签名方式在理论方面可行，但在实际应用中容易受到流氓密钥攻击。目前抵抗流氓密钥攻击主要有两种方式：第一种是要求每个签名者证明它拥有与公钥对应的私钥^[64-66]，此种方式实现较为复杂；第二种方式是要求待签名的消息都是不同的，即验证者拒绝对相同的消息进行签名。

表 2-3 BLS 聚合签名参数

参数	简述
pk_i	用户 u_i 的公钥
msg_i	消息 msg
σ_i	用户 u_i 针对 msg_i 的签名信息
σ	聚合签名
e	双线性映射
g	乘法循环群 G 的生成元

BLS 签名算法实现门限签名很方便。基于 BLS 实现门限签名可分为密钥生成、签名和验证三个步骤。BLS 门限签名的参数说明如表 2-4 所列。

密钥生成阶段由密钥生成中心完成，具体步骤如下：

(1) 首先密钥生成中心随机选取 $x \in \mathbb{Z}_p$ ，使得系统主私钥 $MSK=x$ ，计算系统主公钥 MPK ，系统主公钥的计算如式(2-16)：

$$MPK=g_1^x=v \quad (2-16)$$

(2) 随机选取 $\mathbb{Z}_p$ 上的一个 $t-1$ 阶多项式 P ，满足 $P(0)=x$ ，然后分别计算签名者 u_i 的私钥为 x_i ，公钥为 v_i ， x_i 的计算方法如式(2-17)， v_i 的计算方法如式(2-18)：

$$x_i = P_i \quad (2-17)$$

$$v_i = g_1^{x_i} \quad (2-18)$$

(3) 公开 MPK 和所有用户的公钥。

签名阶段：用户 u_i 使用私钥 x_i 对消息 msg 签名，然后将签名 σ_i 进行全网广播。用户 u_j 收到 u_i 的签名后，首先验证签名 σ_i 的正确性，验证通过则保存收集。待收集到 t 个不同用户的正确的签名后，计算完整的聚合签名。

验证阶段：验证聚合签名是否正确即为验证等式(2-19)是否成立：

$$e(\sigma, g_1) = e(H(msg), MPK) \quad (2-19)$$

表 2-4 BLS 门限签名参数

参数	简述
G_1	阶为 P 的乘法循环群
g_1	G_1 的生成元
e	双线性映射
$H_0: \{0,1\}^* \rightarrow \mathbb{G}_1$	安全 hash 函数
P	$t-1$ 阶多项式
MSK	系统主私钥
MPK	系统主公钥
x_i	签名者 u_i 的私钥
v_i	签名者 u_i 的公钥
σ_i	用户 u_i 针对 m_i 的签名信息

BLS 签名算法的优点有：1) 避免签名者之间的多余通信；2) 聚合签名时，不需要随机数生成器；3) 实现 (p,q) 门限签名容易；4) BLS 签名的长度小于 Schnorr，是 Schnorr 或 ECDSA 的一半。

2.3 秘密分享

秘密分享算法涉及数字签名碎片分发的的问题，主要体现门限签名的实现部分。秘密分享技术本质上是秘密的拆管理。假设将秘密拆分为 m 份，每一份秘密称为秘密碎片，聚合 $n(n < m)$ 份秘密碎片便可以恢复出秘密，允许最多 $n-m$ 份秘密碎片数据丢失，在一定程度上提供了容错性。这个秘密可以是私钥，也可以扩展成其他任意信息，如资产共同管理，谜语答案，秘密遗嘱等。

比较典型的秘密分享算法有 Shamir 秘密分享算法^[67-68]和 Feldman 秘密分享算法^[69]。下面将分别介绍两种秘密分享算法。

2.3.1 Shamir 秘密分享

Shamir 秘密分享算法的核心为拉格朗日插值法，Shamir 使用一元多次多项式来隐藏秘密，一元多次多项式如式(2-20)：

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_{m-1}x^{m-1} \quad (2-20)$$

其中， a_0 即是隐藏起来的秘密，一元多次多项式最高次数和恢复秘密时需要的秘密碎片个数有关。

Shamir 秘密分享算法可分为秘密分割和秘密恢复两个步骤。假设系统内有 n 个节点，且需要至少 m 个密钥碎片才可恢复秘密。秘密分割过程为首先随机生成 a_1 到 a_{m-1} 的系数值。然后在这条度为 $m-1$ 次的曲线上，任意取 n 个不同的点 $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ ，其中 $m \leq n$ ，将这些点的信息分配给 n 个节点，每个节点得到的信息便是秘密的一部分。选取 n 个点而不是整整 m 个点，在一定程度上保证了系统的容错性，可保证在至多 $m-n$ 个节点不提供对应的秘密碎片的情况下依旧可恢复出秘密。密钥恢复过程，至少 m 个用户提供自己专属的秘密碎片后开始计算秘密。计算秘密的过程即为将 m 个点的坐标代入式(2-17)以计算出 $a_0, a_1, \dots, a_{m-1}$ 的值，其中 a_0 就是要恢复的秘密。由于 $m-1$ 个点是随机生成的，通过这些点的曲线，在实数域理论上会有无数条，因此在少于 m 个节点贡献秘密碎片的情况下，概率上不会泄漏出关于 a_0 的任何有用信息。

Shamir 密钥分享方案实现 (m, n) 门限签名，存在两个问题：第一，密钥分发者知晓完整的密钥，有作恶的可能，例如对部分秘密持有者发放错误的分片数据；第二，秘密碎片的持有者可能提供非真实的分片数据。当考虑存在不诚实参与方的情况，对于一个秘密需要有相应的算法来验证其确实是秘密的有效片段。

2.3.2 Feldman 秘密分享

为解决 Shamir 的问题，可验证密钥分享 (Verifiable Secret Share, VSS) 应运而生。其中，Feldman 提出的方案被广为应用。Feldman 的方案针对 Shamir 秘

密分享方案存在的两个问题给出了对应的解决方案：1) 为了解决秘密拥有者作恶的问题，秘密所有者除了给出秘密碎片外，还要提供对应的承诺($c_0, c_1, \dots$)；2) 为解决秘密碎片拥有者作恶的问题，在聚合秘密时，不仅验证秘密碎片，还要验证该用户对应的承诺。

Feldman 秘密分享算法的参数说明如表 2-5 所列。VSS 方案可分为秘密分发阶段和秘密重构阶段。

秘密分发阶段可分为如下步骤：

(1) 秘密所有者选定一个一元 $k-1$ 次多项式，其中， a_0 代表秘密， k 为大于 1 的正整数，多项式的最高次数和秘密聚合时所需的秘密碎片的份数相关。多项式如式(2-21)：

$$a(x) = a_0 + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1} \quad (2-21)$$

(2) 计算承诺。承诺的计算方法如式(2-22)：

$$c_0 = g^{a_0}, c_1 = g^{a_1}, \dots, c_{k-1} = g^{a_{k-1}} \quad (2-22)$$

(3) 秘密所有者广播承诺($c_1, c_2, \dots, c_{k-1}$)给所有参与者，然后单独将秘密碎片 s_i 发送给参与者，此处的密钥碎片为点(x_i, y_i)的信息；

(4) 当第 i 个参与者，收到秘密拥有者发来的数据碎片后，需要验证该秘密碎片的正确性，验证过程即为验证等式(2-23)是否成立：

$$g^{s_i} = \prod_{j=0}^{k-1} (c_j)^{x_i^j} \bmod p \quad (2-23)$$

表 2-5 Feldman 秘密分享算法参数

符号	描述
P	大素数（公开）
q	$p-1$ 的大素数因子（公开）
g	属于 Z_p^* ，且为 q 阶元素（公开）
n	参与者总数
s	秘密
s_i	用户 i 拥有的密钥碎片
k	门限值

由于承诺绑定了系数，如果秘密拥有者发出承诺不是用多项式方程真实系数，会导致验证失败，此举杜绝了秘密拥有者作恶的可能。秘密重构阶段，当一个参与者 i 提供保存的秘密碎片 s_i 和承诺 c_i 时，其他参与者会做同样的验证，这样可以保证参与者在恢复阶段的诚实行为。

2.4 数据聚合

数据聚合算法在区块链中体现为签名碎片的收集和聚合。本文设计的 HSP

共识机制中，主节点收集副本签名发来的签名碎片，对签名碎片聚合之后，得到聚合签名。该聚合签名可看作各阶段信息的投票结果，数据聚合的方式可直接影响系统的性能。

目前，数据聚合有多种方式。传统方式为中心化聚合，在聚合之初，首先要选择一个中心节点，然后由中心节点完成数据的收集和聚合。目前应用较广的方式是采用树形结构进行数据传输，进而完成数据聚合。下面介绍中心化聚合和二叉交流树聚合。

2.4.1 中心化聚合

在众多节点中，选择一个中心节点 **major**，由 **major** 完成发送、接收并聚合数据的任务，此种方式称为中心化合传输。如图 2-2 所示，中心化合传输的优点是在节点数量较少的情况下，完成聚合的时间短。但随着节点数量的增多，聚合完成时间会线性增加。另外一个较大缺陷是当系统中某一个节点故障而导致数据传输失败时，整个系统需要重新传输，否则会造成数据丢失。

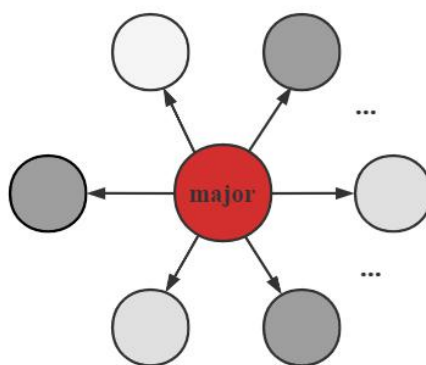


图 2-2 中心化聚合

2.4.2 二叉交流树

二叉交流树的思想源于 Antony Rowstron 和 Peter Druschel 于 2001 年发表的 Pastry^[70] 论文。在 Pastry 网络中，每个节点都有一个唯一的标识符(**nodeId**)。当有消息需要传输的时候，当前消息所在的节点会有效地将消息路由到当前所有活跃的节点中 **nodeId** 在数字上最接近当前 **nodeId** 的节点。每个节点在 **nodeId** 空间中记录它的近邻，并在新节点加入、节点故障和恢复时进行全网广播。Pastry 完全分散、可伸缩、自组织，自动适应节点的到达、离开和故障。在多达 10 万个节点的模拟网络上通过原型实现获得的实验结果证实了 Pastry 的可扩展性和容错性。

二叉交流树^[71]的概念于 2008 年由 Justin Cappos 和 John H. Hartman 提出。在

二叉交流树中，每个节点通过重复地与其他节点交换数据来创建自己的二叉树，理想情况下，系统内节点存储的数据都是一样的，任一节点故障对最终的结果不会造成影响。节点聚合出完整数据的复杂度为 $O(\log_2 n)$ 。

二叉交流树的原理可以概述为：每个节点首先与其节点 ID 只有最低有效位不同的节点交换数据，然后与节点 ID 次低有效位不同的节点交换数据，直到与节点 ID 只有最高有效位不同的节点交换数据。如图 2-3 所示，系统中存在 8 个节点，8 个节点均构造属于自己的二叉交流树，理想情况下，8 个节点各自创建的二叉交流树是完全相同的。

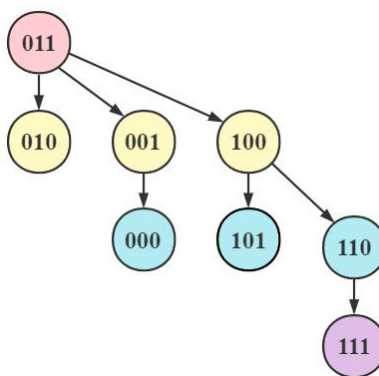


图 2-3 二叉交流树

二叉交流树的优点是容错性强，在节点数量多的情况下扩展性强；但也存在缺点，让每个节点创建自己的二叉交换树会产生过多的开销。

2.5 共识机制

所谓共识机制，也就是通过节点的共同投票，在短时间内实现对交易的检验与确认。而对于同一个交易，一旦利益不相干的若干个节点能取得共识，便可以认为对全网的此笔交易已经形成共识。

本文的研究重点为联盟链，联盟链的典型共识机制为拜占庭容错类共识机制。目前比较典型的拜占庭容错类共识机制有 PBFT、SBFT 和 Hotstuff 等，下面介绍这三种算法。

2.5.1 PBFT

PBFT 采用了两轮点对点通信使得系统中各节点的状态达成一致。PBFT 将系统内的所有节点分为主节点和副本节点，主节点负责打包消息，然后将打包后的消息传输给副本节点，副本节点收到主节点发来的消息，开启共识流程。

图 2-4 为系统未出现视图切换时的区块共识流程。PBFT 算法达成共识需要

三个阶段：Pre-Prepare、Prepare 和 Commit 阶段。

(1) Pre-Prepare 阶段：主节点接收来自客户端的请求并验证，验证通过后将请求封装为 pre-prepare 消息并全网广播给副本节点。

(2) Prepare 阶段：副本节点接收到主节点发来的 pre-prepare 消息，验证消息的合法性，验证通过后全网广播 prepare 消息给其他副本节点，此副本节点全网广播消息的同时会接收来自其他副本节点的消息，当收到至少 $2f+1$ 个 prepare 消息后，此节点进入到 prepared 状态。

(3) Commit 阶段：副本节点进入到 prepared 状态后，全网广播 commit 消息，当收到至少 $2f+1$ 个 commit 消息后，此节点进入到 committed 状态，发送 reply 消息给客户端。客户端收到至少 $f+1$ 个一致的 reply 消息后，可以确认其发出的请求已被执行。

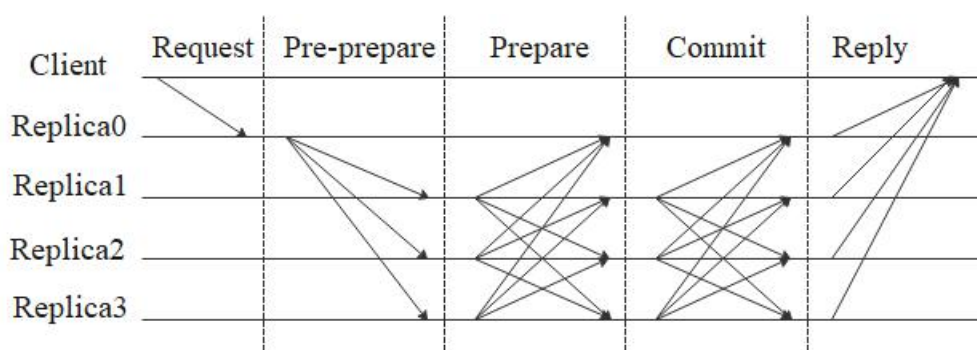


图 2-4 PBFT 共识流程

当出现大范围的网络故障或主节点为拜占庭节点时，系统会出现视图切换。所谓视图，可理解为一个节点作为主节点进行共识的过程。每个视图对应一个唯一的视图编号，在此视图期间，由该主节点对客户端发来的请求进行验证处理，验证通过后对请求进行排序编号，最后将请求打包成区块并全网广播。当主节点故障或系统网络故障等意外情况发生时，可引发视图切换，视图切换成功后，视图编号加 1。

以主节点故障为例介绍视图切换流程。视图切换过程涉及的相关参数如表 2-6 所列。当主节点发生故障，副本节点可发起视图切换请求。副本节点在认为主节点为拜占庭节点时全网广播视图切换消息 $\langle \text{view-change}, v+1, \text{num}, C, P \rangle$ ，此时此副本节点停止接收除视图切换消息和稳定检查点消息之外的一切消息。当节点收到足够多的视图切换请求后，首先判断自己是否为下一任主节点，如果是，广播 $\langle \text{new-view}, v+1, V, O \rangle$ 消息给其他副本节点，开始下一视图，如果不是，则等待接受 new-view 消息。在发生视图切换时，系统的通信复杂度为 $O(n^3)$ ，其中 n 为系统内节点数。

表 2-6 PBFT 视图切换过程相关参数

参数	简述
v	当前视图编号
v+1	新（下一任）视图编号
num	稳定检查点编号
C	$2f+1$ 个节点验证过的检查点的消息集合
P	序列号大于 num 但没有回复给客户端的请求的集合
V	收到的视图切换请求的集合
O	预先准备的消息

PBFT 的缺点有：1) 系统内节点数量大时，系统性能较差；2) 当有 1/3 及以上的节点为拜占庭节点时，系统不能完成共识，不能正常运作；3) 不支持共识节点的动态加入和退出。

针对 PBFT 算法存在的问题，出现了多种改进形式，如 BFT-SMART^[72]、SBFT 和 HoneyBadgerBFT^[73]等。

2.5.2 SBFT

大多数拜占庭容错系统仅支持小规模集群，当节点数量增多时，系统的吞吐量会大幅度下降，共识延迟时间延长。于是，Guy Golan Gueta 等人于 2019 年提出 SBFT 算法。SBFT 使用收集器技术^[50]来减少通信量，将通信复杂度从 $O(n^2)$ 级别降到多项式级别。使用门限签名技术减少收集器消息规模和验证签名的计算开销。当所有节点均正常运行时，SBFT 允许使用快速共识机制，当有节点出现故障时，可以在快速共识和正常共识之间无缝切换。

SBFT 的快速共识流程如图 2-5 所示，SBFT 将系统内节点分为四类，分别为 Primary、c-collector、e-collector 和 replica。SBFT 达成快速共识需要五个阶段，依次为 Pre-prepare、Sign-share、Full-commit-Proof、Sign-state 和 Full-execute-Proof 阶段。快速共识的五个阶段的具体工作如下：

(1) Pre-prepare 阶段：Primary 接收到客户端的请求，当收到至少 b 个请求后，将这些请求打包成区块后广播 pre-prepare 消息给所有的 replicas，pre-prepare 消息中包含打包好的区块和 Primary 针对此区块的签名等信息。

(2) Sign-share 阶段：replica 收到 pre-prepare 消息后对消息验证，通过验证后对 pre-prepare 消息签名，将签名消息包含在 sign-share 消息中，发送 sign-share 消息给 C-Collector。

(3) Full-commit-proof 阶段：C-Collector 对收集到的 sign-share 消息进行检查，当接收到若干个不同的 sign-share 消息后，将这些消息组合成一个聚合签名，

将聚合签名包含在 full-commit-proof 消息中，C-Collector 发送 full-commit-proof 消息给其他 replicas。replica 收到 full-commit-proof 消息后，如果通过验证，则 commit 此区块。

(4) Sign-state 阶段：当 replica-i 收到 commit 的区块后，replica-i 执行区块中的 request 并更新状态，然后更新状态摘要，并对摘要进行签名，将签名消息包含在 sign-state 消息中，并将 sign-state 消息发送给 E-Collectors。

(5) Full-execute-proofs 阶段：E-Collector 收集 sign-state 消息并验证签名，当收到 $f+1$ 个时，组成一个签名并将签名包含在 full-execute-proof 消息中，发送 full-execute-proof 消息给所有的 replica。replica 收到 full-execute-proof 消息，并检查 full-execute-proof 消息中的签名，收集到一定数量的签名后为该区块创建一个 full-execute-proof 消息，然后广播 full-execute-proof 消息给所有的副本节点和 client。full-execute-proof 消息的意义是告诉副本节点和客户端当前状态是持久的并且 request 已经被执行。

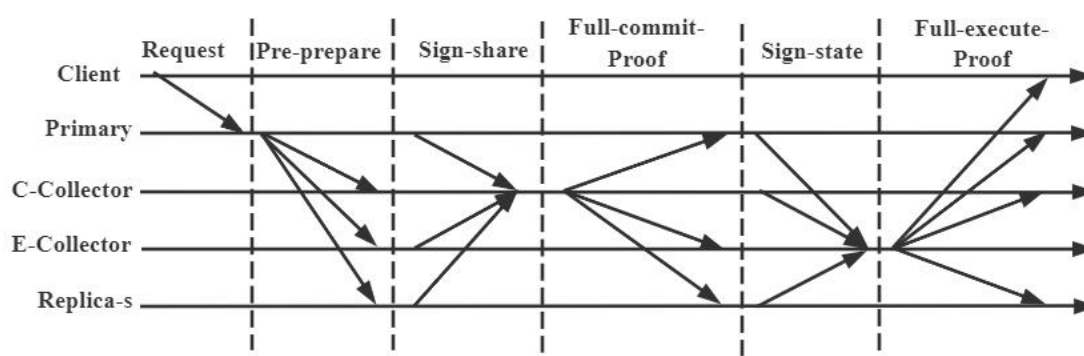


图 2-5 SBFT 快速共识流程

当无法达成快速共识时，使用线性 PBFT，其中线性 PBFT 是对 PBFT 的改编，使用了门限签名和线性通信。当主节点出现故障或系统出现网络故障时，副本节点发起视图切换请求，视图切换流程类似 PBFT 的视图切换流程，此处不予赘述。

SBFT 缺点有：1) 在出现视图切换时，通信复杂度仍为 $O(n^3)$ ；2) 有类似于 Paxos 的集中化的趋势；3) 不支持共识节点的动态加入和退出。

2.5.3 Hotstuff

Hotstuff 在节点间通信方面，采用星型拓扑结构；采用门限签名技术，再次降低节点间的通信复杂度；采用三轮确认的方式，降低了系统在进行视图切换时的复杂度，Hotstuff 每完成一次共识，就会更换主节点，进入到下一个视图；同时，还提出了流水线共识的概念，流水线共识可简化共识流程，降低共识延迟。

下面对 Hotstuff 共识流程介绍。

如图 2-6 所示, Hotstuff 算法达成共识需要四个阶段: Prepare 阶段、Pre-Commit 阶段、Commit 阶段、Decide 阶段。四个阶段的具体工作如下:

(1) Prepare 阶段: 新任主节点收集由 $2f+1$ 个副本节点发送的 new-view 消息, 其中 f 为任意正整数。new-view 消息中包含了此节点上高度最高的 prepareQC, prepareQC 是共识过程中第一阶段确认达成的证据。主节点从收到的 new-view 消息中, 选取高度最高的 preparedQC 作为 highQC, 主节点广播 prepare 消息, prepare 消息中包含了新的区块和 highQC, highQC 作为 prepare 消息的安全性验证。副本节点收到当前视图编号对应的主节点的 prepare 消息, 副本节点对 prepare 消息进行验证, 验证通过后, 对 prepare 消息签名, 将签名发送给主节点。

(2) Pre-Commit 阶段: 主节点收集签名, 收集到至少 $2f+1$ 个关于该 prepare 消息的签名后进行签名聚合操作, 聚合后的签名作为 prepareQC 保存在本地, 然后将其放入 pre-commit 消息中广播给副本节点。副本节点收到 pre-commit 消息, 进行验证, 验证通过后, 对 pre-commit 消息签名, 将签名发送给主节点。

(3) Commit 阶段: 主节点收集至少 $2f+1$ 个关于 pre-commit 消息的签名后进行签名聚合操作, 聚合签名作为 precommitQC, 并将其放在 commit 消息中广播。副本节点收到 commit 消息, 进行验证, 验证通过后, 对 commit 消息签名, 然后回复给主节点。此时副本节点锁定 precommitQC, 将本地的 lockQC 更新成收到的 precommitQC。

(4) Decide 阶段: 当主节点收到至少 $2f+1$ 个关于 commit 消息的签名, 聚合成 commitQC, 广播 decide 消息。副本节点收到 decide 消息中的 commitQC 后, 执行区块中的交易, 同时视图编号加 1, 开始新的视图。

客户端收集到多于 $f+1$ 个一致的 decide 消息后, 可以认为其发出的请求被执行。

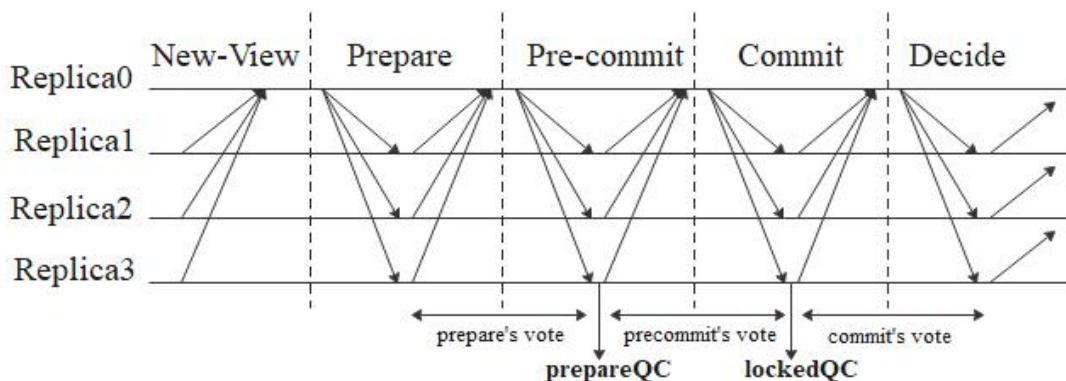


图 2-6 Hotstuff 正常共识流程

可以看出，Hotstuff 的三个确认阶段具有相同的结构，即副本节点对消息进行投票，主节点收集投票信息并将投票结果广播至其他节点，这些阶段可以被统一表示并流水线化。如图 2-7 所示，流水线在每个 Prepare 阶段进行视图更改，也就是说 v4 视图的关于 b4 的 prepare 阶段的投票也相当于 b3 的 pre-commit 阶段的投票，相当于 b2 的 commit 阶段的投票，相当于 b1 的 decide 阶段投票，复用了不同阶段的投票信息，简化了共识的消息交互轮次。

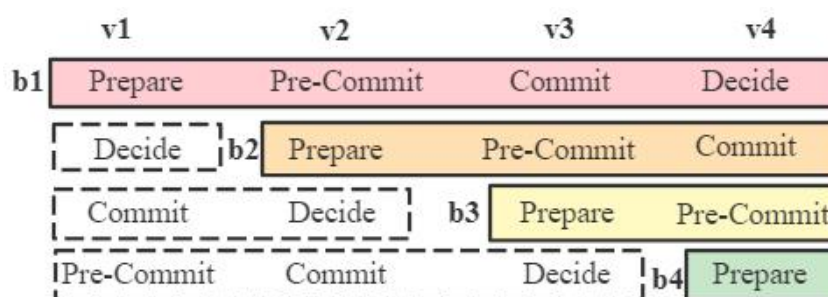


图 2-7 Hotstuff 流水线共识

当主节点发生故障时，副本节点发起视图切换请求至下一任主节点，下一任主节点 id 的计算方式由用户自行设定，视图切换请求中需要包含此副本节点内最高的 prepareQC。下一任主节点收集视图切换请求，统计视图切换请求中的 prepareQC 信息，基于 prepareQC 中最高的试图编号选出一个 highQC，并基于此 highQC 构造新块，继续共识。因为 highQC 是视图编号最大的，所以不会有比它更高的区块得到确认，这样新创建的区块肯定不会和之前的区块冲突。在发生视图切换时系统的通信复杂度为 $O(n)$ 。

Hotstuff 的缺点有：1) secp256k1 签名算法在实现聚合签名和门限签名方面，效率较低；2) 主节点工作压力大，需要完成接受请求、验证请求、打包请求、发送区块、收集签名、验证签名和聚合签名的任务；3) 如果采用流水线式三轮确认方式，一个区块的确认需要等待四个视图。

2.6 本章小结

本章主要介绍了区块链和共识机制的相关知识。首先介绍了区块的组成，然后介绍了数字签名算法，详细的介绍了 ECDSA、Schnorr 和 BLS 三种数字签名算法的密钥生成、签名及签名验证过程，同时也介绍了三种算法实现聚合签名和门限签名的步骤。紧接着介绍了秘密分享算法，分别介绍了 Shamir 秘密分享算法和 Feldman 可验证秘密分享算法。接着详细的介绍了中心化数据聚合方式和二叉交流树数据聚合方式。最后介绍了实用拜占庭容错机制和几种以其为基础的改

进机制，详细的介绍了 **SBFT** 共识机制和 **Hotstuff** 共识机制。这些理论知识为第三章 **HSP** 共识机制提供了理论依据。

第三章 HSP 共识机制设计

针对联盟链共识机制存在主节点压力大、扩展性差和视图切换复杂的问题，本章提出了 HSP 共识机制。为降低主节点的压力，我们采用了 BLS 门限签名算法；为解决拓展性差的问题，我们使用了改进的二叉交流树机制；为降低系统进行视图切换时的通信复杂，我们采用三阶段确认方式。

3.1 BLS 门限签名

门限签名的目的是为了解决节点间通讯复杂度高的问题，签名的流程为首先使用 BLS 签名算法生成公私钥，之后用 VSS 传输给用户。

本章以 $(2k+1, 3k+1)$ 门限签名为例，介绍 BLS 门限签名算法流程。如图 3-1 所示，应用在区块链系统中的 BLS 门限签名算法主要由公私钥初始化、消息签名、聚合签名和聚合签名验证四部分组成。下面将详细介绍这四个部分的具体实现。

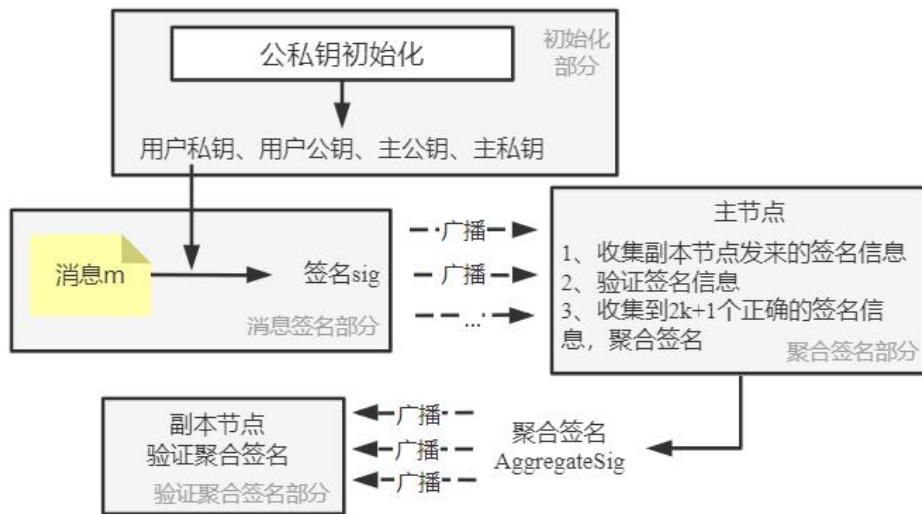


图 3-1 BLS 门限签名流程

3.1.1 公私钥初始化

系统用户的密钥均由密钥生成中心生成，BLS 数字签名参数如表 3-1 所列。密钥生成步骤如下：

(1) 密钥生成中心选择任一随机数 $x \in \mathbb{Z}_p$ ，主私钥 $MSK = x$ ，主公钥 $MPK = v$ ，主公钥的计算如式(3-1)：

$$v = g_2^x \in G_2 \quad (3-1)$$

(2) 密钥生成中心随机选择 $\mathbb{Z}_p$ 上的 $2k$ 阶多项式，多项式系数为 $a_0, a_1, \dots, a_{2k}$ ，

满足 $P(0) = x$ ，之后开始计算承诺 c_i ，承诺的计算如式(3-2)：

$$c_i = g_2^{a_i} \quad (3-2)$$

(3) 承诺计算完毕后，由密钥生成中心向系统内节点广播 $\{c_0, c_1, c_2, \dots, c_{2k}\}$ ；

(4) 计算签名者（或称系统内节点）的公私钥，签名者 u_i 的私钥为 x_i ，公钥为 v_i 。私钥 x_i 的计算公式如式(3-3)所示，公钥 v_i 的计算如式(3-4)：

$$x_i = P(i) \quad (3-3)$$

$$v_i = g_2^{x_i} \in G_2 \quad (3-4)$$

(5) 密钥生成中心将公钥和私钥传输给对应的签名者，签名者收到承诺 $\{c_0, c_1, c_2, \dots, c_{2k}\}$ 和公私钥后，需要对公私钥进行验证，防止在公私钥在传输过程中被篡改。验证过程即为验证式(3-5)是否成立，如果成立，则记录自己的公私钥。

$$g_2^{y_i} = c_0 c_1^{x_i} c_2^{x_i^2} \dots c_{m-1}^{x_i^{m-1}} \quad (3-5)$$

(6) 公开 MPK 和所有用户的公钥。

表 3-1 BLS 数字签名参数说明

符号	描述
G_1, G_2	阶为 P 的乘法循环群
g_1	G_1 的生成元
g_2	G_2 的生成元
e	双线性映射函数
$h: \{0,1\}^* \rightarrow G_1$	Hash 函数

3.1.2 消息签名

用户 u_i 用私钥 x_i 对消息 m 进行签名，计算签名 σ_i ，然后将签名 σ_i 和消息 msg 广播，签名的计算方法如式(3-6)：

$$\sigma_i = h(msg)^{x_i} \quad (3-6)$$

3.1.3 聚合签名及验证

聚合签名的步骤如下。用户 u_j 收到用户 u_i 的签名 σ_i 和消息 msg ：

(1) 首先将消息 msg 进行 Hash 运算，然后验证签名 σ_i 的正确性。签名验证即验证等式(3-7)是否成立，若验证通过，则记录此签名：

$$e(\sigma_i, g_2) = e(h(msg), v_i) \quad (3-7)$$

(2) 待收集到 $2k+1$ 个不同用户的正确签名后，计算完整签名 σ 。签名的计算方式如式(3-8)：

$$\sigma = \prod_{i=1}^{2k+1} \sigma_i^{\lambda_i}, \text{ 其中 } \lambda_i = \frac{\prod_{j=1, j \neq i}^{2k+1} (0-j)}{\prod_{j=1, j \neq i}^{2k+1} (i-j)} \pmod{p} \quad (3-8)$$

由拉格朗日插值公式可知，任意 $2k+1$ 个用户产生的完整签名相同。

多个签名经过运算后，得到聚合签名。聚合签名的正确性也是我们关注的问题，因此需要对聚合签名进行验证。聚合签名验证的步骤如下：

(1) 首先验证承诺 c_0 是否正确，承诺 c_0 的计算公式如式(3-9)：

$$c_0 = g_2^{a_0} \quad (3-9)$$

(2) 如果等式(3-9)成立，则继续验证聚合签名和主公钥的双线性映射配对是否成功。验证过程即为验证等式(3-10)是否成立：

$$e(\sigma, g_2) = e(h(msg), MPK) \quad (3-10)$$

3.2 改进二叉交流树

分根节点可分为善意节点和恶意节点，不同性质的分根节点对应不同的聚合方式，所以对传统的二叉交流树进行了改进。改进方式分为系统内不存在恶意节点和分根节点为恶意节点两种情况。

3.2.1 不存在恶意节点的信息交换方式

传统的二叉交流树算法，系统内的节点均会根据节点编码和节点间的物理距离创建一棵完整的二叉交流树，若系统内存在 n 个节点，理想情况下会产生 n 棵二叉交流树，即每个节点都需要维护一棵完整的二叉交流树。如果将二叉交流树技术直接应用在区块链系统中，存在两个问题：1) 系统中的每一个节点都需要维护一个二叉树，对于性能较好的节点，此行为可能不会增加节点负担，但是不适用于轻量级节点；2) 并不是所有的节点都有聚合全部数据的需求，因此并不是每个节点都需要维护一颗完整的二叉交流树。

针对该问题做如下改进：将系统内节点分组，主根节点在每个小组内部选择一个节点与其进行数据交换，被选择的节点作为小组的分根节点，重复相同的分组操作、数据交换操作。

改进后的二叉交流树算法流程的核心为：根节点每次只与每个小组内的一个节点进行信息交换，同根节点进行信息交换的节点 id 随机选择，然后分根节点作为小组内的根节点再次对小组内的节点进行分组和数据交换操作。如图3-2中，主根节点 011 首先同 010 节点进行信息交换，再同 001 节点进行信息交换，最后同 101 节点进行信息交换。Group3 中，101 节点作为分根节点，首先进行分组，分组后首先同 100 节点进行信息交换，之后在 110 节点和 111 节点随机选择一个节点进行第二次的信息交换，图 3-2 中 101 节点选择 110 节点。

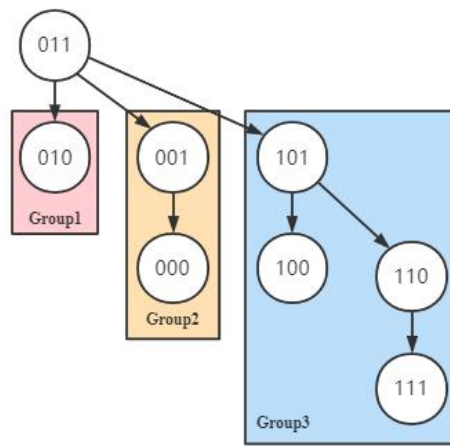


图 3-2 无恶意节点的改进二叉交流树

算法实现部分：系统内节点的 id 默认为十进制表示，需要将节点 id 全部转换为二进制，下面所述节点 id 均为二进制数，算法实现步骤如下：

(1) 将系统总节点数转换为二进制数表示，记为 $count$ ， $count$ 的长度为 len bit，则可将系统内节点分为 len 组；

(2) 分组。第一组，第一组内有个 2^0 节点，此节点 id 同主节点 id (记 $primaryId$) 只有最后一位不同；第二组，第二组内有 2^1 个节点，此小组内的节点 id 的前 $len-2$ 位同 $primaryId$ 一致，倒数第二位同 $primaryId$ 不一样，倒数第一位任意；第三组，第三组内有 2^2 个节点，此小组内的节点的前 $len-3$ 位同 $primaryId$ 一致，倒数第三位 $primaryId$ 相反，最后两位任意，因此此小组有四个节点，……，第 len 组，此小组内有 2^{len-1} 个节点，第一位和 $primaryId$ 不一致，其他 $len-1$ 位随意。此时，完成第一次分组；

(3) 根节点从每个小组中随机选择一个节点作为此小组的根节点。小组内的根节点再次进行分组，分组流程类似第二步，一直迭代，直到形成二叉交流树。子节点同其父节点进行信息交换，最终主节点获得完整数据。

以上为系统内节点总数为 2^t 个 (t 任意正整数)。当系统内节点总数不满足 2^t 个时，此时必定会出现某一个或者某几个小组不满员的情况，此种情况下，不满员的小组内部依旧按照分组规则进行分组，不会影响最终聚合结果。

3.2.2 分根节点为恶意节点的信息交换方式

3.2.1 节描述的改进的二叉交流树的算法流程得以实现的基础为分根节点均为非故障节点，而在实际应用中，需要考虑到分根节点是故障节点的情况。

分根节点是故障节点的处理方式如下：如果分根节点为故障节点，此时需要根据主根节点收集到的节点信息的数量来判断是否需要继续收集节点信息。当根

主节点收集到的信息数量大于等于指定值，则忽略故障节点，继续进行到下一阶段。如果根主节点收集到的信息数量小于指定值，则根主节点发送请求至故障节点的子节点，请求内容为要求分节点发送节点信息至根主节点。分节点收到请求后需要对请求进行验证，验证请求涉及多个方面，如验证发送该请求的节点是否为主节点。请求通过节点的验证后，该节点将请求发送至主根节点。此时主根节点又一次统计此时的信息数量，并判断是否再次进行信息收集，如此循环。假设图 3-3 中 011 节点需要收集到至少 5 条信息且 101 节点故障，此时 011 节点收到 3 条信息，小于指定值，因此 011 节点 100 节点和 110 节点发送请求，100 节点和 110 节点收到请求后，验证 011 节点的身份，验证通过后将信息发送至 011 节点。在拜占庭容错类系统中，若拜占庭节点数为 f ，一般取指定值为 $2f+1$ 。

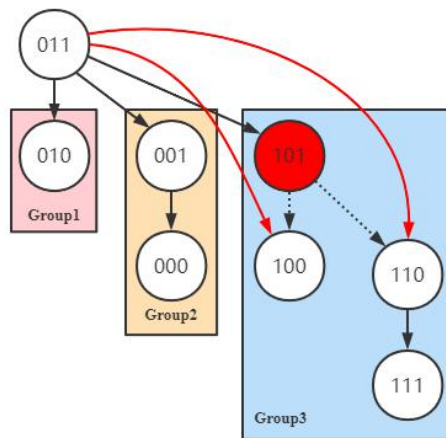


图 3-3 分根节点故障的改进二叉交流树

3.3 HSP 共识流程

HSP 共识机制的共识流程如图 3-4 所示。客户端将请求发送至主节点，主节点验证请求后，将一定数量的请求打包成区块，对区块进行签名之后将区块信息发送至副本节点。副本节点收到区块信息后，对信息进行验证和投票，将投票信息发送至指定节点，指定节点再一级级向上传输，最终传输到主节点。主节点收集到一定数量的投票信息后，推进共识进入到下一阶段。

整个共识流程，涉及到数字签名，签名验证，签名聚合，主节点选择和共识细节。3.1 小节和 3.2 小节详细的描述了数字签名的签署、聚合和验证部分，下面将描述主节点的选择和具体的共识细节部分。

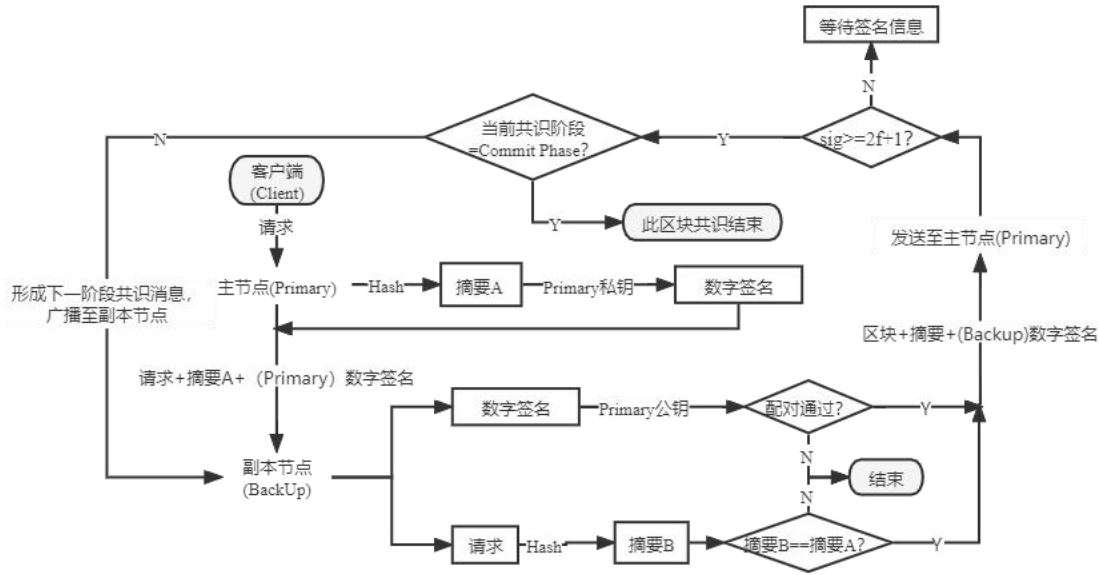


图 3-4 HSP 共识机制共识流程

3.3.1 主节点选择

系统主节点的选择方式有许多种，最简单的选择方式是通过预定公式计算主节点的 ID，较为常见的计算公式如式(3-11)：

$$LeaderID = viewNumber \div counts \quad (3-11)$$

其中 $viewNumber$ 为当前视图编号， $counts$ 为系统内节点总数。但此种方式存在缺陷，即每一轮的主节点 ID 是可预知，此种情况下攻击者可以通过攻击特定的节点来使得系统陷入频繁的视图切换过程，进而使得系统无法处理交易，最终使得系统瘫痪。因此，此种主节点 ID 生成方式在共识机制性能模拟时使用较多。本设计在进行模拟实验的采用的便是此种方式。

目前实际应用中使用较为广泛的主节点选择方式是可验证随机函数^[73-74] (Verifiable Random Function, VRF)。VRF 要求生成的数是随机的，并且要求参与者能够验证该数的生成，即保证生成的数具有随机性和可验证性。

3.3.2 HSP 正常共识流程

正常的共识流程如图 3-5 所示，取系统内节点总数为 $3f+1$ ，拜占庭节点数小于等于 f ，HSP 算法达成共识需要四个阶段：Prepare、Pre-Commit、Commit 和 Decide 阶段。

当主节点为非拜占庭节点时，主节点接受来自客户端的请求，并对该请求进行验证，验证的内容包括时间戳、客户端 ID 和客户端签名等信息，请求验证通过后被打包成区块 A。在 Prepare 阶段，主节点全网广播 prepare 消息，prepare 消息中包括区块 A 和节点 ID，此处的节点 ID 为该副本节点的签名信息发送至的

节点的 ID。如图 3-5 所示，节点 11 将签名信息发送至节点 10，节点 10 聚合节点 11 和自身的签名信息后，发送至节点 00，节点 01 将签名信息发送至节点 00。在 Pre-Commit 阶段，主节点计算收集的签名数量，主节点收集到大于等于 $2f+1$ 个关于 prepare 消息的签名信息后对签名进行聚合，聚合后的签名作为 prepareQC 保存到本地并将其放在 pre-commit 消息中广播给副本节点。副本节点验证 pre-commit 消息，验证通过后对该消息进行签名，并将签名发送给指定节点。指定节点聚合签名后，将聚合签名最终传输至主节点，此时节点处于 prepared 状态。Commit 阶段，主节点收集到大于 $2f+1$ 个关于 pre-commit 消息的签名后对签名进行聚合，聚合后的签名作为 lockedQC（也称 precommitQC）并放在 commit 消息中进行全网广播。副本节点收到 commit 消息后进行验证，验证通过后对 commit 消息签名，将签名发送给指定节点，最终签名传输至主节点，此时节点处于 pre-committed 状态。Decide 阶段，主节点收到至少 $2f+1$ 个关于 commit 消息的签名后，聚合成 commitQC，全网广播 decide 消息。副本节点收到 decide 消息中的 commitQC 后，执行区块中的交易，并将执行结果依次返回至客户端，此时节点处于 committed 状态。客户端收集到多于 $f+1$ 个节点的一致的 decide 消息后，认为其发起的请求被执行。

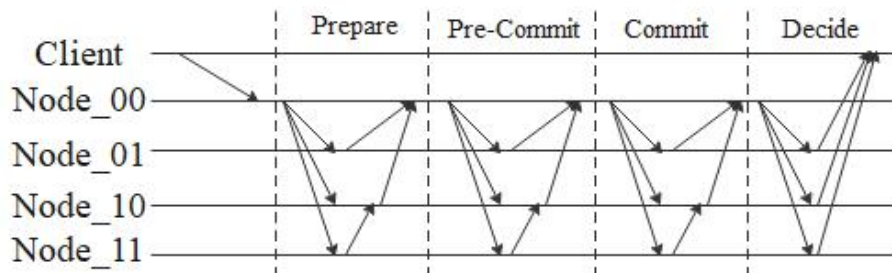


图 3-5 HSP 正常共识流程

3.3.3 HSP 视图切换流程

系统出现视图切换主要是两种情况：（1）主节点为拜占庭节点；（2）系统内大部分节点发生网络故障。

HSP 的视图切换流程如图 3-6 所示，当副本节点认为主节点故障时，副本节点发送视图切换请求 $\langle \text{view-change}, \text{replicaID}, \text{timestamp}, \text{prepareQC} \rangle$ 至新任主节点，新任主节点的 ID 号由算法计算得来。下一任主节点收集视图切换请求，统计视图切换请求中的 prepareQC 信息，基于 prepareQC 中最高的试图编号选出一个 highQC，并基于此 highQC 构造新块。highQC 是视图编号最大的，所以不会有比它更高的区块得到确认，这样新创建的区块肯定不会和之前的区块冲突，同时主节点广播 $\langle \text{new-view}, \text{replicaID}, \text{timestamp}, \text{highQC} \rangle$ 消息至全网副本节点。副本节

点接收到 new-view 消息后进入新的视图。此时客户端发送请求至新的主节点，新任主节点收集请求并开始新一轮的共识。

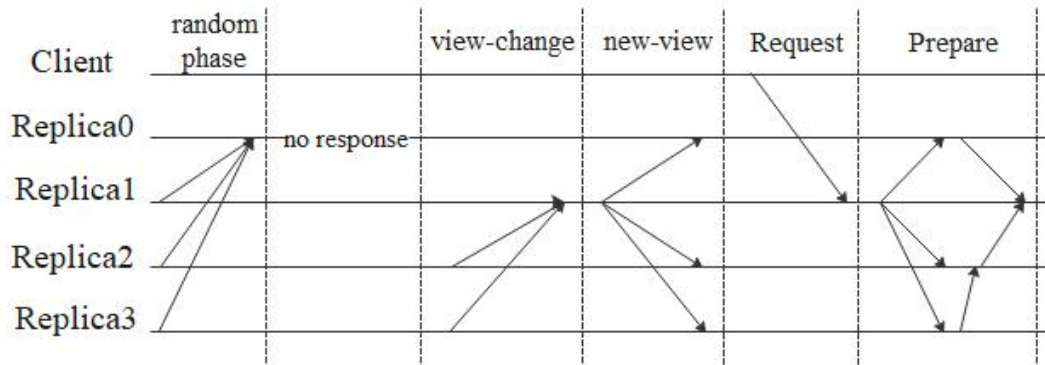


图 3-6 HSP 视图切换流程

3.4 安全性分析

区块链系统的基本安全目标是指数据安全、智能合约安全、隐私保护和共识安全。一般采用 CIA 原则来描述数据安全，即保密性（Confidentiality）、完整性（Integrity）和可用性（Availability）。数据安全主要体现在三个方面，第一信息内容非交易双方不可知，第二信息完整且未篡改，第三信息可获取，需要考虑密钥泄露、51%攻击、贿赂攻击和自私挖矿攻击等问题。智能合约安全体现在合约的编写安全和运行安全，需要考虑重入攻击、整数溢出攻击、交易顺序依赖攻击和时间戳依赖攻击等，避免如 DAO 事件和美链事件等类似事件的发生。隐私保护体现在身份隐私保护和交易隐私保护，如需要考虑如何保证交易双方的个人信息不被泄露和交易的具体内容非交易双方不可知问题。共识机制是区块链系统正常运行的核心，共识机制决定了系统内无利益相关的节点如何就一个内容达成一致。达成一致的过程的细节和流程关系到系统性能，共识机制的安全直接关系到系统能否正常运作，因此共识安全尤为重要。

本章的研究重点为区块链共识机制，因此重点分析共识安全，采用一致性和活性两个安全属性来衡量和评估区块链的共识安全^[72]。

一致性要求任何经过共识后成功上链的交易均无法被回滚，也可理解为攻击者无法通过有效手段使得区块链分叉，从而使得冲突的区块同时被诚实节点确认。一致性是共识机制最重要的安全目标。区块链系统在发生视图切换时，容易出现分叉情况，HSP 针对冲突区块的处理类似 Hotstuff。假设系统内节点数为 $3f+1$ ，拜占庭节点数为 f ，针对区块 B 若有小于 $2f+1$ 个节点处于 prepared 状态，则换掉 B，如果大于等于 $2f+1$ 个节点处于 prepared 状态或更高状态，则新区块的高度高于区块 B。此种策略可保证一致性。

活性要求诚实节点提交的合法交易在一段时间后终将会被永久的记录在区块链上。由 CAP(Consistency, Availability, Partition)定理可知,在完全异步的区块链网络中,一致性和活性是不可能同时达成的。假设网络长时间的大规模的失去了同步,此种情况下,在“无法进行交易,但是资金不会丢失”和“可以进行交易,但不保证收到的资金以后可用”之间,所有人都会选前者,可见对于区块链共识机制而言,应该优先保证一致性,再保证活性。在实际应用中,会对“活性”加限制来保证“一致性”。因此,HSP 算法的应用场景设置为部分同步系统。所谓部分同步系统即消息在网络中传输时有延迟上限,即消息一定会送到,但延迟上限未知。若消息传输延迟大于设定上限,则节点可发起视图切换请求来更换主节点,重新开始消息传输,保证了系统活性。

3.5 本章小结

本章为 HSP 共识机制的核心部分,详细的阐述了 HSP 共识机制中所采用的各项技术及实现过程。首先介绍了 BLS 门限签名算法的实现细节,详细的介绍了消息签名过程、聚合签名过程和验证聚合签名的过程。紧接着对传统二叉交流树进行分析,分析出其直接应用在区块链系统中存在的问题,并针对相应问题给出了对应的解决方式。考虑到系统中存在故障节点的情况,首先给出了无拜占庭节点的情况下数据聚合的算法实现流程,针对出现故障节点的情况也介绍了在出现故障节点的情况下的信息聚合方式。然后对 HSP 的共识流程进行了详细介绍,主要从以下几方面进行介绍:1)介绍了主节点的两种选择方式及其适用场景;2)介绍了在没有发生视图切换的情况下的共识流程,详细的阐述了共识四个阶段每个阶段需要做的事情;3)介绍了在发生视图切换时系统的共识流程。本章的最后从一致性和活性两个角度对 HSP 共识机制的安全性进行了分析。

第四章 HSP 共识机制测试及结果分析

HSP 共识机制在数字签名算法和数据交换方式两个层面对 Hotstuff 共识机制进行了改进。控制数字签名算法和数据交换方式两个变量，对 HSP 共识机制的吞吐量和共识延迟进行测试。同时，考虑改进对系统安全性的影响，设计了视图切换实验，测试系统在视图切换时的吞吐量和共识延迟。最后，HSP 共识机制摒弃了 Hotstuff 的共识流水线化的思想，使得区块确认不需要等待四个视图，避免了频繁的视图切换流程，进一步提升系统性能。理想情况下，HSP 共识机制较 Hotstuff 相比，吞吐量会有一定范围的提升，共识延迟会有适当的下降。

4.1 实验环境

使用 java 语言完成算法的编写，使用 Docker 容器技术模拟搭建多节点的区块链环境，设定区块链环境的网络为部分同步网络。搭建测试环境的操作系统为 64 位 Windows10 专业版，CPU 为 Intel(R) Core(TM) i5-6200U CPU @ 2.30GHz (4 CPUs), ~2.4GHz，内存为 16GB，Docker 版本为 19.03.12，Java 的 JDK 版本为 1.8。

4.2 实验设计

实验设计思想：HSP 共识机制在两个层面对 Hotstuff 共识机制进行了改进：1) 数字签名算法方面，使用 BLS 数字签名算法取代 ECDSA 数字签名算法，需要设计实验来分析 BLS 数字签名算法对系统性能的影响；2) 在数据交换方式上，理论上改进的二叉交流树的技术可在一定程序上提升聚合数字签名验证效率和系统的可扩展性，因此，需要验证改进的二叉交流树技术对系统性能的影响。同时，考虑到系统的一致性和活性，需要人为的设计一定数量的拜占庭节点，来模拟出现视图切换的情况，需要测试系统在出现视图切换情况下，系统能否正常的完成共识过程以及计算分析此种情况下系统的吞吐量和共识延迟。

根据上述分析，设计了如下三组实验：

第一组实验：取拜占庭节点数为 0，节点总数为 4，测试 request/reply 为 0/0，256/256，512/512 三种情况下 Hotstuff+ECDSA 算法、Hotstuff+BLS 算法的吞吐量和共识延迟。本实验控制的变量为数字签名算法。实验目的：测试 BLS 数字签名算法对 Hotstuff 算法的影响。补充说明，节点数量不会影响 BLS 签名算法的表现，原因为：BLS 数字签名算法是对消息进行签名、打包和验证处理，其性能的好坏表现为单位时间内处理交易的数量，和节点数量无关。

第二组实验：拜占庭节点数为 0，系统内节点总数为 4、7、10、16、64 五种

情况下 Hotstuff (BLS) 算法和 HSP 算法的吞吐量和共识延迟, 针对不同的 request/reply, 重复做三次实验。其中, Hotstuff (BLS) 为使用 BLS 签名算法代替 ECDSA 签名算法的 Hotstuff 共识机制, HSP 为使用了 BLS 签名算法和改进二叉交流树技术的改进共识机制, 即本实验控制的变量为改进的二叉交流树技术, 二叉交流树技术主要影响数字签名聚合效率和系统的可扩展性。实验目的: 测试节点数量对 HSP 算法和 Hotstuff (BLS) 算法的影响。

第三组实验: 取系统内节点总数依次为 4、7、10、16、64, 人为制造网络故障和节点故障, 故障节点数不得超过节点总数的 1/3, 针对不同的 request/reply 做两组实验, 测试在发生视图切换时, Hotstuff+BLS 算法和 HSP 算法的吞吐量和共识延迟。实验目的: 1) 测试由于网络故障引起的视图切换对系统的性能的影响以及此种情况下系统的一致性和活性; 2) 测试主节点连续故障引起的连续视图切换对系统的性能的影响以及系统一致性和活性。

4.3 实验结果展示及分析

4.3.1 数字签名实验结果及分析

第一组实验的测试结果如图 4-1、图 4-2 所示。从整体上看, 可以得出两个结论。第一个: 随着 request/reply 的变大, 系统的吞吐量降低, 共识延迟加长, 此种情况属于正常现象。第二个: 使用 BLS 签名算法的 Hotstuff 性能优于使用 ECDSA 签名算法的 Hotstuff。从细节上分析, 在系统中拜占庭节点数为 0, 总节点数为 4 的情况下, 当 request/reply=256/256 时, Hotstuff+BLS 较 Hotstuff+ECDSA 吞吐量提升了 19.5%, 共识延迟降低了 19.9%; 当 request/reply=512/512 时, Hotstuff+BLS 较 Hotstuff+ECDSA 吞吐量提升了 22.6%, 共识延迟降低了 17.3%。

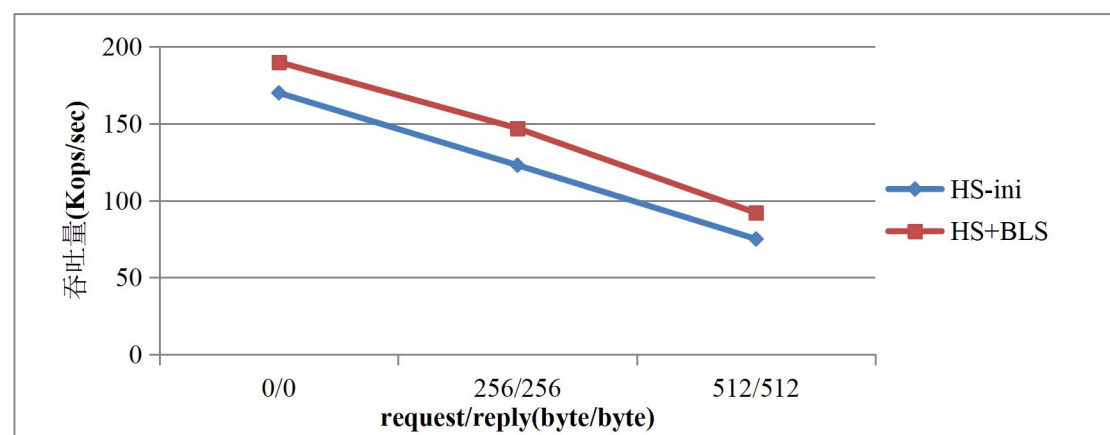


图 4-1 吞吐量对比

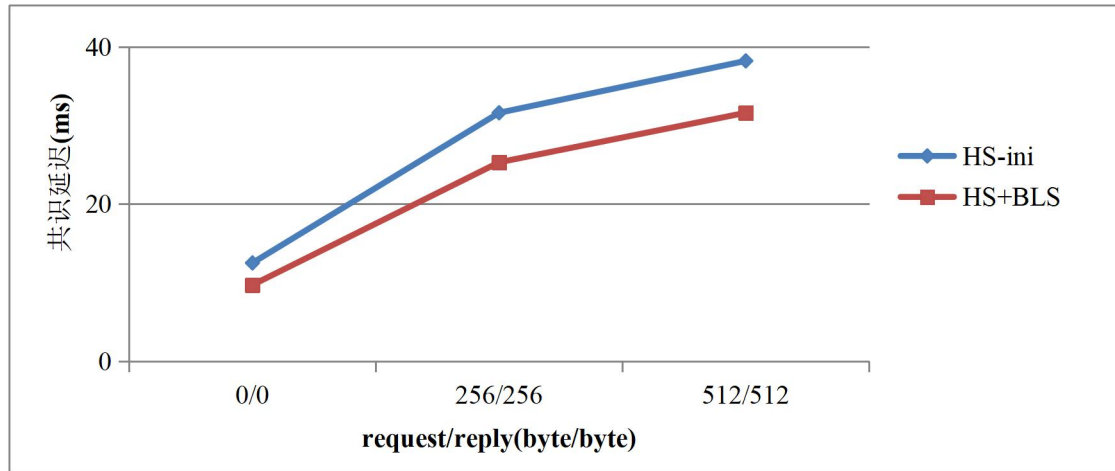


图 4-2 共识延迟对比

出现上述实验结果的原因如下：1) request/reply 变大，相当于需要处理的信息量变大，必定会在一定程度上降低系统的性能。2) BLS 签名算法较 ECDSA 算法相比，实现交易批量验证和门限签名较为容易，使得吞吐量增加的同时共识延迟减小。

4.3.2 二叉交流树实验结果及分析

第二组实验的测试结果如图 4-3、图 4-4 所示。从整体上看，在节点数量较少的情况下，Hotstuff+BLS 算法的表现优于 HSP 算法，即二叉交流树的引入对于增长吞吐量、降低系统延迟没有明显的作用。但随着节点数量增多，HSP 算法和 Hotstuff+BLS 算法的差距越来越小。从细节上看，在系统内拜占庭节点为 0，节点总数为 10 时，当 request/reply=256/256 时，HSP 算法较 Hotstuff+BLS 算法吞吐量降低了 0.8%，共识延迟下降了 11.2%；当 request/reply=512/512 时，HSP 算法较 Hotstuff+BLS 算法吞吐量下降了 5.2%，共识延迟下降了 3.0%。节点总数为 16 时，当 request/reply=256/256 时，HSP 算法较 Hotstuff+BLS 算法吞吐量降低了 7.0%，共识延迟下降了 2.8%；当 request/reply=512/512 时，HSP 算法较 Hotstuff+BLS 算法吞吐量提升了 11.1%，共识延迟下降了 6.9%。在节点数量为 64 时，当 request/reply=256/256 时，HSP 算法较 Hotstuff+BLS 算法吞吐量提升了 33.8%，共识延迟下降了 16.4%；当 request/reply=512/512 时，HSP 算法较 Hotstuff+BLS 算法吞吐量提升了 69.0%，共识延迟下降了 11.2%。

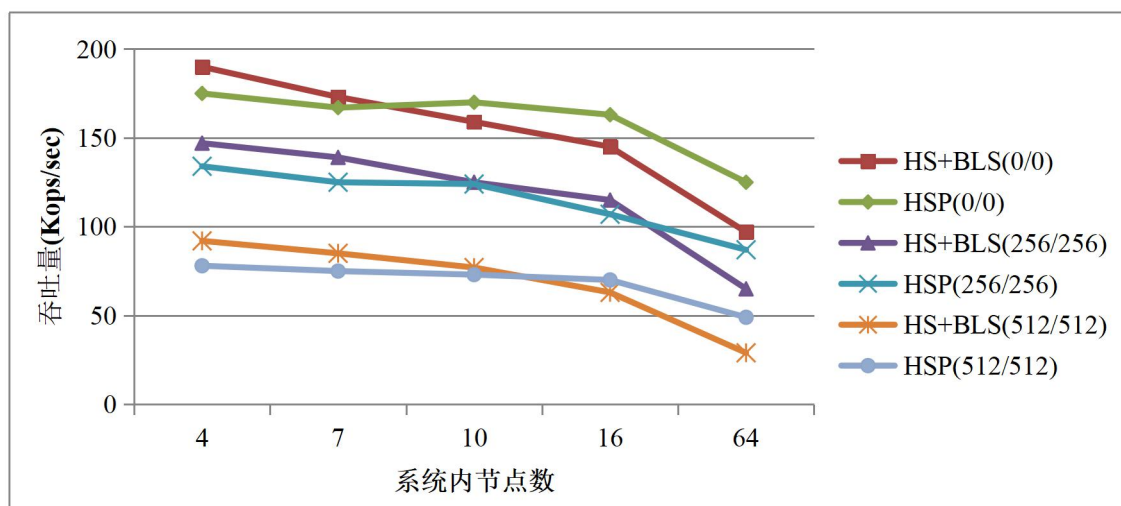


图 4-3 吞吐量对比

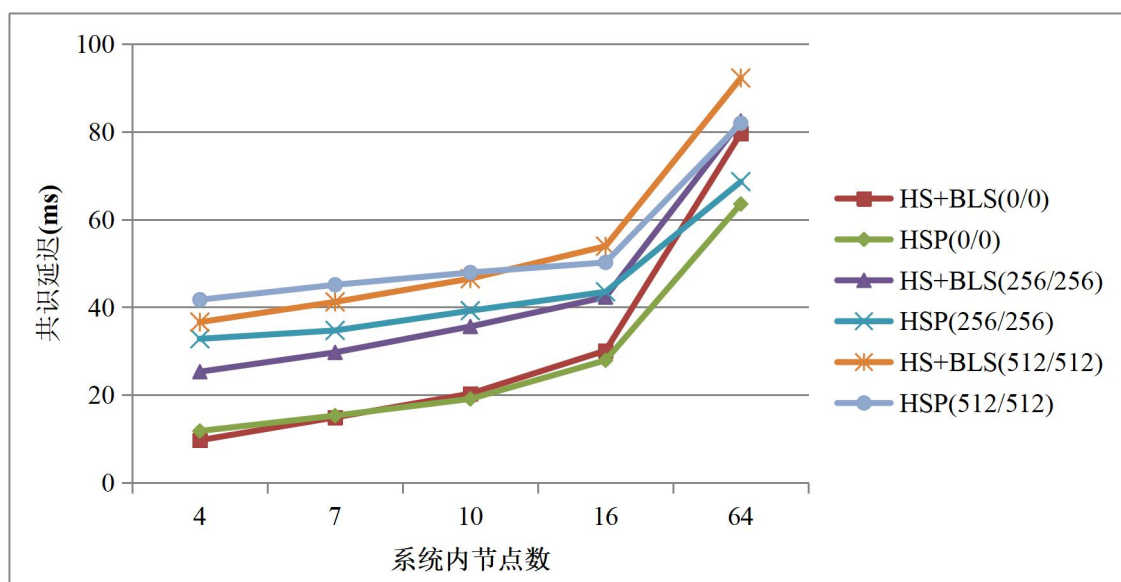


图 4-4 共识延迟对比

出现上述实验结果的原因如下：1) 节点数量较少时，HSP 算法首先需要对节点进行分组操作，分组操作会消耗一定时间。2) 节点数量多时，由于主节点不是同所有的副本节点进行信息交换，只是同每个小组内的一个成员进行信息交换，此时主副节点通信次数降低。3) 分根节点会聚合其子节点的签名，降低了主节点聚合全部签名的压力，缩短了共识时长。

4.3.3 视图切换实验结果及分析

第三组实验的测试结果如图 4-5、图 4-6 所示。从整体上看，在出现视图切换时，较无视图切换时系统的性能有所下降；同时，较系统网络故障相比，主节点故障对系统性能的影响更大；最重要的一点是，当系统出现视图切换时，系统依

旧能够保持一致性和活性。从细节上看,在节点总数为 10 且 request/reply=256/256 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量降低了 5.4%,共识延迟增加了 1.5%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量降低了 6.5%,共识延迟增大了 1.9%。在节点总数为 10 且 request/reply=512/512 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量降低了 9%,共识延迟增大了 3.8%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量降低了 10.8%,共识延迟增大了 1.7%。在节点总数为 16 且 request/reply=256/256 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量提升了 3.8%,共识延迟下降了 10.0%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量提升了 15.6%,共识延迟降低了 6.1%。在节点总数为 16 且 request/reply=512/512 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量提升了 15.6%,共识延迟下降了 7.7%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量没有变化,共识延迟降低了 3.5%。在节点总数为 64 且 request/reply=256/256 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量提升了 35.1%,共识延迟下降了 22.6%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量提升了 34.4%,共识延迟降低了 17.9%。在节点总数为 64 且 request/reply=512/512 时,在由于网络故障而发生视图切换的情况下,HSP 算法较 Hotstuff 算法吞吐量提升了 38.3%,共识延迟下降了 11.1%;在由于主节点连续故障而连续进行视图切换的情况下,HSP 算法较 Hotstuff+BLS 吞吐量提升了 183.0%,共识延迟降低了 12.6%。

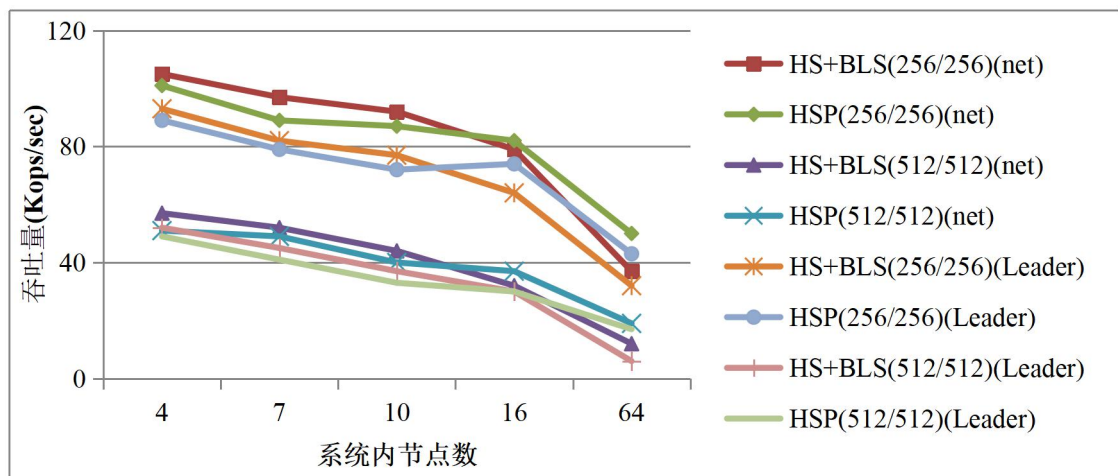


图 4-5 视图切换时吞吐量对比

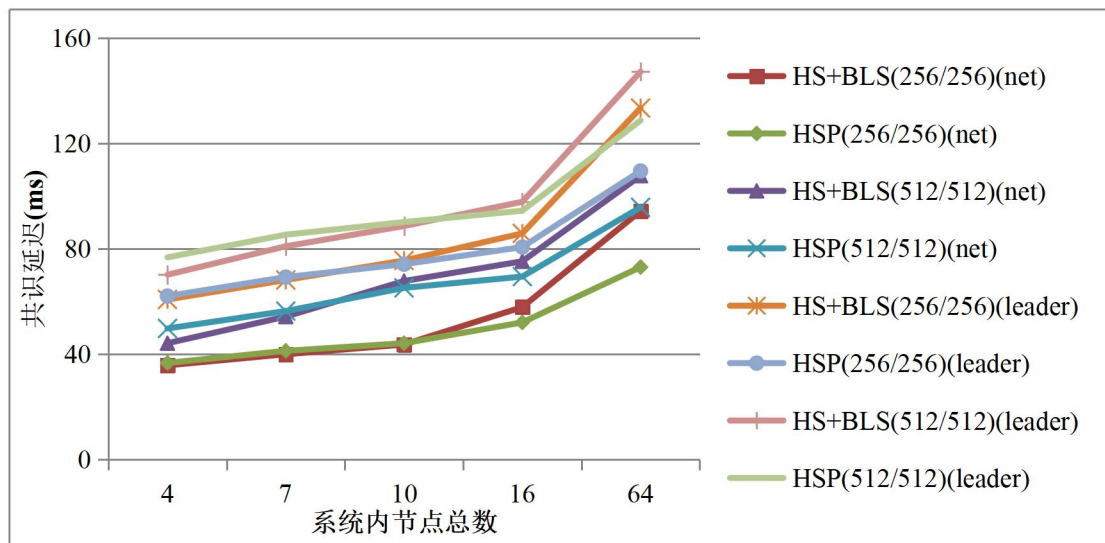


图 4-6 视图切换时共识延迟对比

出现上述实验结果的原因如下：1) 当出现视图切换时，系统性能降低属于正常情况，原因是视图切换过程会消耗时间。2) 主节点故障对系统性能的影响更大的原因可理解为新任主节点为拜占庭节点可引发视图连环切换，视图切换时，系统内节点不处理交易，因此吞吐量降低、共识延迟加长。3) 当系统内拜占庭节点数量小于总节点数的 $1/3$ 时，系统针对拜占庭节点的处理方式可保证系统正常运行。

4.4 本章小结

本文在 Hotstuff 算法的理论基础上提出了 HSP 共识机制，主要的创新点在于替换了数字签名算法、引入了二叉交流树技术和采用了三阶段投票共识流程。本章首先介绍了实验环境，介绍了算法编写的语言、CPU 型号和 Docker 版本等。然后根据创新点分别设计了三组实验，第一组实验是测试 BLS 签名算法对系统性能的影响，第二组实验是测试二叉交流树对系统性能的影响，第三组实验是测试出现视图切换时，系统的一致性和活性。紧接着，对实验结果进行了展示并对实验结果进行了分析，实验结果表明：1) BLS 数字签名算法较 ECDSA 数字签名算法在提升系统性能方面起到了积极作用；2) 在系统节点数量较少的情况下，HSP 算法的优势较采用了 Hotstuff+BLS 较弱；随着节点数量的增加，HSP 共识机制的优势逐渐体现，吞吐量随着节点数量的增加，下降的慢，且共识延迟也未随着节点数量的增多而大幅度增长；3) 在出现视图切换时，系统的性能有所下降，但均可以保证共识过程的正确运行。

第五章 总结与展望

5.1 总结

区块链作为数字时代的最前沿技术,已经延伸到数字金融、物联网和供应链管理等领域。目前,区块链技术不断的从数字货币向非金融领域渗透扩散,联盟链的成员认证服务、可插拔式共识机制和系统性能良好等特性均优于公有链,均使得联盟链具有广阔的应用前景。截止到 2020 年底,区块链技术在产品溯源、智慧医疗和智慧政务等方面均有应用落地,区块链产业蓬勃发展。因此,研究区块链技术对于促进社会进步具有重要意义。

区块链中的密码学技术为区块链的匿名性、防篡改、可验证的特性提供了保证,共识机制保证了区块链系统的正常运行,使得系统内交易得以正确、快速上链。随着区块链技术的不断发展,研究人员对于共识机制的研究也未曾中断。如何提高交易的吞吐量,如何降低共识延迟,如何提高系统的可扩展性,如何提高共识机制的效率等,一直以来都是研究热点。

目前针对拜占庭类共识机制的改进方向主要有:1) 数字签名算法方向,即使用效率更高、性能更优的算法;2) 数据交换方式方向,如最初的 P2P,再到星型拓扑结构等;3) 限制参与共识的节点数量,参与共识的节点数量越少,理论上达成共识的速度越快,但安全性也随之降低,去中心化程度也随共识节点数量的减少而降低;4) 针对系统内节点的状态,采取不同的共识策略;5) 优化视图切换流程等。

本文对区块链技术和共识机制的历史和发展现状均做了详细分析,研究了多种应用于联盟链的共识机制,最后从数字签名、数据交换方式和视图切换流程三个方面对 Hotstuff 共识机制进行了优化。对数字签名算法、聚合签名技术、门限签名技术、数据聚合技术和阶段共识等区块链涉及的技术进行了研究后,提出了 HSP 共识机制。本文详细描述了 HSP 中采用的 BLS 数字签名算法和改进的二叉交流树技术的实现过程,并详细的阐述了 HSP 的共识流程,并对 HSP 进行了安全性分析,最后对 HSP 进行了模拟测试,并对测试结果进行了分析。测试结果显示,在相同的测试环境下,HSP 算法较 Hotstuff 算法相比,吞吐量有不同程度的提升,共识延迟有不同程度的下降。本文主要成果如下:

(1) HSP 共识机制使用 BLS 数字签名算法代替 Hotstuff 中采用的 ECDSA 数字签名算法。BLS 签名的体积小于 ECDSA,使用 BLS 签名使得区块可以容纳更多的交易。BLS 签名算法实现交易批量验证较 ECDSA 相比难度较低,加快了区块的验证速度。使用 BLS 签名算法实现门限签名,降低了通信复杂度,增强了系统的可扩展性。测试结果表明,在系统中拜占庭节点数为 0,总节点数为 4

的情况下, 当 request/reply=256/256 时, Hotstuff+BLS 较 Hotstuff+ECDSA 吞吐量提升了 19.5%, 共识延迟降低了 19.9%。

(2) 对二叉交流树技术进行了改进, 采用改进的二叉交流树技术进行数字签名聚合。数字签名在部分子节点处便可进行聚合, 只有最后一次数字签名聚合发生在主节点, 将主节点的压力下放至副本节点, 降低主节点的压力。同时考虑到分节点是拜占庭节点的情况, 在分节点故障时依旧可以完成指定数量的数字签名聚合。测试结果表明, 在系统中拜占庭节点数为 0, 总节点数量为 64 的情况下, 当 request/reply=256/256 时, HSP 算法较 Hotstuff+BLS 算法吞吐量提升了 33.8%, 共识延迟下降了 16.4%。

(3) 区别于传统的两阶段确认方式, 本设计为三阶段确认方式, 将系统发生视图切换时的通信复杂度降到多项式级别。传统的拜占庭容错类共识机制进行视图切换的通信复杂度为 $O(n^3)$, HSP 共识机制在系统出现视图切换的情况下的通信复杂度为 $O(n)$, 在节点数量多的情况下, 极大的降低视图切换时的通信复杂度。同时, 摒弃 Hotstuff 的流水线共识设计, 避免了频繁的视图切换流程, 进一步提升系统表现。测试结果表明, 在节点总数为 64 且 request/reply=256/256 时, 在由于网络故障而发生视图切换的情况下, HSP 算法较 Hotstuff 算法吞吐量提升了 35.1%, 共识延迟下降了 22.6%; 在由于主节点连续故障而连续进行视图切换的情况下, HSP 算法较 Hotstuff+BLS 吞吐量提升了 34.4%, 共识延迟降低了 17.9%。

5.2 展望

本文对联盟链的共识机制进行研究, 设计出了通信复杂度低、视图切换简单、节点压力分散以及系统规模易于拓展的 HSP 共识机制, 得到了较好的测试结果, 但仍存在不足之处, 后续研究可以从以下几点进行:

(1) 在实际应用中, 区块链系统内的节点数量应当是动态变化的, 如比特币。一个性能良好的共识机制应当保证系统内节点数量在动态变化的情况下, 依旧可以保证共识达成, 即系统支持节点的动态加入和退出。本文提出的 HSP 共识机制, 不具备动态性, 即节点数量在系统启动时是固定的。后续工作需要进一步的优化设计, 在保证吞吐量和共识延迟的基础上, 实现动态性。

(2) 对于拜占庭节点没有惩罚措施。后续工作考虑对拜占庭节点设置在固定时间内禁止加入系统作为惩罚, 对于出现多次故障的节点, 禁止加入系统。

(3) HSP 算法在节点数量较多时表现优于 Hotstuff, 但在节点数量较少时表现略差, 因为改进的二叉交流树技术早期进行分组会消耗时间。在后续优化设计时, 可考虑在节点数量少的情况下, 采用一种其他的共识机制, 节点数量多的情况下, 采用 HSP 共识机制。

参考文献

- [1] 区块链技术发展现状及趋势[J]. 检察风云, 2021(03):32-33.
- [2] NAKAMOTO S. Bitcoin: A peer-to-peer electronic cash system[J]. Decentralized Business Review, 2008: 21260.
- [3] 袁勇, 王飞跃. 区块链技术发展现状与展望[J]. 自动化学报, 2016, 42(4): 481-494.
- [4] BANO S, SONNINO A, AL-BASSAM M, et al. SoK:Consensus in the age of blockchains[C]. Proceedings of the 1st ACM Conference on Advances in Financial Technologies, New York: ACM, 2019:183-198.
- [5] ZHANG R, PRENEEL B. Lay down the common metrics:Evaluating proof-of-work consensus protocols' security[C]. Proceedings of 2019 IEEE Symposium on Security and Privacy, Piscataway: IEEE, 2019: 175-192.
- [6] WAN S, LI M, LIU G, et al. Recent advances in consensus protocols for blockchain: a survey[J]. Wireless networks, 2020, 26(8): 5579-5593.
- [7] 袁勇, 倪晓春, 曾帅等. 区块链共识算法的发展现状与展望[J]. 自动化学报, 2018, 44(11):2011-2022.
- [8] KING S, NADAL S. Ppcoin: Peer-to-peer crypto-currency with proof-of-stake[J]. self-published paper, 2012,19.
- [9] LARIMER D. Delegated proof-of-stake (dpos)[J]. Bitshare whitepaper, 2014, 81: 85.
- [10]SOMPOLINSKY Y, ZOHAR A. Secure high-rate transaction processing in bitcoin[C]. Proceedings of the 2015 International Conference on Financial Cryptography and Data Security, Cham: Springer, 2015: 507-527.
- [11]LAMPORT L. The Part-Time Parliament[J]. ACM Computing Surveys, 1998, 16(2):133-169.
- [12]LAMPORT L. Paxos Made Simple[J]. ACM SIGACT News, 2001, 32(4):18-25.
- [13]LAMPORT L, MASSA M. Cheap paxos[C]. Proceedings of International Conference on Dependable Systems and Networks, Piscataway: IEEE, 2004: 307-314.
- [14]CHANDRA T D, GRIESEMER R, REDSTONE J. Paxos made live: an engineering perspective[C]. Proceedings of the 26th annual ACM symposium on Principles of distributed computing, New York: ACM, 2007:398-407.
- [15]ONGARO D, OUSTERHOUT J. In search of an understandable consensus algorithm[C]. Proceedings of 2014 {USENIX} Annual Technical Conference, Berkeley: USENIX, 2014:305-319.
- [16]CASTRO M, LISKOV B. Practical Byzantine fault tolerance and proactive recovery[J]. ACM Transactions on Computer Systems, 2002, 20(4): 398-461.
- [17]孙一蓬. 基于联盟链的多链式区块链共识性能研究[D]. 东华理工大学, 2019.

- [18]王海勇, 郭凯璇, 潘启青. 基于投票机制的拜占庭容错共识算法[J]. 计算机应用, 2019, 39(06):1766-1771.
- [19]GUETA G G, ABRAHAM I, GROSSMAN S, et al. SBFT: a scalable and decentralized trust infrastructure[C]. Proceedings of the 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, Piscataway: IEEE, 2019:568-580.
- [20]YIN M, MALKHI D, REITER M K, et al. Hotstuff: Bft consensus with linearity and responsiveness [C]. Proceedings of 2019 ACM Symposium on Principles of Distributed Computing. New York: ACM, 2019: 347-356.
- [21]ABRAHAM I, MALKHI D, NAYAK K, et al. Sync Hotstuff: Simple and practical synchronous state machine replication[C]. Proceedings of the 2020 IEEE Symposium on Security and Privacy. Piscataway: IEEE, 2020:106-118.
- [22]李娟娟, 袁勇, 王飞跃. 基于区块链的数字货币发展现状与展望[J]. 自动化学报, 2021, 47(04):715-729.
- [23]BUTERIN V. A next-generation smart contract and decentralized application platform[J]. white paper, 2014, 3(37).
- [24]FUJIMURA S, WATANABE H, NAKADAIRA A, et al. BRIGHT: A concept for a decentralized rights management system based on blockchain[C]. Proceedings of 2015 IEEE International Conference on Consumer Electronics-Berlin, Piscataway: IEEE, 2015: 345-346.
- [25]林小驰, 胡叶倩雯. 关于区块链技术的研究综述[J]. 金融市场研究, 2016(02):97-109.
- [26]CHRISTIDIS K, DEVETSIKIOTIS M. Blockchains and Smart Contracts for the Internet of Things[J]. IEEE Access, 2016, 4: 2292-2303.
- [27]DORRI A, KANHERE S S, JURDAK R, et al. Blockchain for IoT security and privacy: The case study of a smart home[C]. Proceedings of 2017 IEEE International Conference on Pervasive Computing and Communications Workshops, Piscataway: IEEE, 2017:618-623.
- [28]金凯. 基于区块链的食品供应链溯源系统设计与实现[D]. 北京工业大学, 2019.
- [29]周秀秀. 基于区块链的食品信息溯源研究[D]. 重庆邮电大学, 2020.
- [30]张国潮, 唐华云, 陈建海等. 基于区块链的数字音乐版权管理系统[J]. 计算机应用, 2021, 41(04):945-955.
- [31]徐龙, 刘勇, 李晓坤. 基于共识机制的电力物联网群组密钥管理研究[J/OL]. 电测与仪表:1-6[2022-01-21]. <http://kns.cnki.net/kcms/detail/23.1202.TH.20210508.1515.007.html>.
- [32]杨德昌,赵肖余,徐梓潇等.区块链在能源互联网中应用现状分析和前景展望[J]. 中国电机工程学报, 2017, 37(13):3664-3671.
- [33]夏清, 窦文生, 郭凯文等. 区块链共识协议综述[J]. 软件学报, 2021, 32(02): 277-299.
- [34]HABER S, STORNETTA W S. How to time-stamp a digital document[C].

- Proceedings of the Conference on the Theory and Application of Cryptography, Berlin: Springer, 1990:437-455.
- [35] HABER S, STORNETTA W S. Secure names for bit-strings[C]. Proceedings of the 4th ACM Conference on Computer and Communications Security, New York: ACM, 1997: 28-35.
- [36] BAYER D, HABER S, STORNETTA W S. Improving the Efficiency and Reliability of Digital Time-Stamping[M]. New York: Springer, 1993.
- [37] MERKLE R C. Protocols for public key cryptosystems[C]. Proceedings of 1980 IEEE Symposium on Security and Privacy, Piscataway: IEEE, 1980:122-134.
- [38] MERKLE R C. A digital signature based on a conventional encryption function [C]. Proceedings of the Conference on the theory and application of cryptographic techniques, Berlin: Springer, 1987:369-378.
- [39] SZYDLO M. Merkle tree traversal in log space and time[C]. Proceeding of the 2004 International Conference on the Theory and Applications of Cryptographic Technique, Berlin: Springer, 2004:541-544.
- [40] DWORK C, NAOR M. Pricing via processing or combatting junk mail[C]. Proceedings of 1992 Annual international cryptology conference, Berlin: Springer, 1992:139-147.
- [41] TROMP J. Cuckoo cycle: a memory bound graph-theoretic proof-of-work[C]. Proceedings of the International Conference on Financial Cryptography and Data Security, Berlin: Springer, 2015: 49-62.
- [42] 王尚阳. 基于区块链的安全高效数据共享方案[D]. 西安电子科技大学, 2019.
- [43] 孙国梓, 王纪涛, 谷宇. 区块链技术安全威胁分析[J]. 南京邮电大学学报(自然科学版), 2019, 39(05):48-62.
- [44] SOMPOLINSKY Y, ZOHAR A. Secure high-rate transaction processing in bitcoin[C]. Proceedings of the International Conference on Financial Cryptography and Data Security, Berlin: Springer, 2015: 507-527.
- [45] EYAL I, GENCER A E, SIRER E G, et al. Bitcoin-ng: A scalable blockchain protocol[C]. Proceedings of the 13th symposium on networked systems design and implementation {NSDI}, Berkeley: USENIX, 2016: 45-59.
- [46] SOMPOLINSKY Y, WYBORSKI S, ZOHAR A. PHANTOM and GHOSTDAG: A Scalable Generalization of Nakamoto Consensus[J]. Cryptology ePrint Archive, 2018.
- [47] SOMPOLINSKY Y, LEWENBERG Y, ZOHAR A. SPECTRE: A Fast and Scalable Cryptocurrency Protocol[J]. IACR Cryptol, 2016, 2016:1159.
- [48] 孙国梓, 李芝, 肖荣宇等. 区块链交易安全问题研究[J/OL]. 南京邮电大学学报(自然科学版), 2021(02):41-53[2022-01-21]. DOI:10.14132/j.cnki.1673-5439.2021.02.006.
- [49] LAMPORT L, SHOSTAK R, PEASE M. The Byzantine generals problem[M]. Concurrency: the Works of Leslie Lamport, New York: ACM, 2019: 203-226.
- [50] KOTLA R, ALVISI L, DAHLIN M, et al. Zyzzyva: speculative byzantine fault

- tolerance[C]. Proceedings of 21th ACM SIGOPS symposium on Operating systems principles, New York: ACM, 2007:45-58.
- [51] GUETA G G, ABRAHAM I, GROSSMAN S, et al. SBFT:a scalable and decentralized trust infrastructure[C]. Proceedings of the 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. Piscataway: IEEE, 2019:568-580.
- [52] 辛运伟, 廖大春, 卢桂章. 单向散列函数的原理、实现和在密码学中的应用[J]. 计算机应用研究, 2002(02):25-27.
- [53] 邵奇峰, 金澈清, 张召等. 区块链技术:架构及进展[J]. 计算机学报, 2018, 41(05): 969-988.
- [54] SZABO N. Formalizing and securing relationships on public networks[J]. First monday, 1997.
- [55] 王川, 孙斌. 数字签名算法及其比较[J]. 信息安全与通信保密, 2008(06):64-66.
- [56] 陈传波, 祝中涛. RSA 算法应用及实现细节[J]. 计算机工程与科学, 2006(09): 13-14+87.
- [57] 王尚平, 王育民, 张亚玲. 基于 DSA 及 RSA 的证实数字签名方案[J]. 软件学报, 2003(03):588-593.
- [58] JOHNSON D, MENEZES A, VANSTONE S. The Elliptic Curve Digital Signature Algorithm (ECDSA)[J]. International Journal of Information Security, 2001, 1(1):36-63.
- [59] 张伟. ECDSA 算法实现及其安全性分析[J]. 信息与电子工程, 2003(02):7-12.
- [60] SCHNORR C.P. Efficient signature generation by smart cards[J]. Journal of Cryptology, 1991, 4(3):161-174.
- [61] BONEH D, LYNN B, SHACHAM H. Short signatures from the Weil pairing[C]. Proceedings of the International conference on the theory and application of cryptology and information security, Berlin: Springer, 2001:514-532.
- [62] BONEH D, DRIJVERS M, NEVEN G. Compact multi-signatures for smaller blockchains[C]. Proceedings of the International Conference on the theory and application of cryptology and information security, Berlin: Springer, 2018:435-464.
- [63] SYTA E, TAMAS I, WISHER D, et al. Keeping authorities" honest or bust" with decentralized witness cosigning[C]. Proceeding of the 2016 IEEE Symposium on Security and Privacy, Piscataway: IEEE, 2016:526-545.
- [64] BOLDYREVA A. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme[C]. Proceeding of the International Workshop on Public Key Cryptography, Berlin: Springer, 2003: 31-46.
- [65] LU S, OSTROVSKY R, SAHAI A, et al. Sequential aggregate signatures and multisignatures without random oracles[C]. Proceeding of Annual International Conference on the Theory and Applications of Cryptographic Techniques, Berlin:

- Springer, 2006: 465-485.
- [66] RISTENPART T, YILEK S. The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks[C]. Proceeding of Annual International Conference on the Theory and Applications of Cryptographic Techniques, Berlin: Springer, 2007: 228-245.
- [67] SHAMIR A. How to share a secret[J]. Communications of the ACM, 1979, 22(11): 612-613.
- [68] BLAKLEY G R.. Safeguarding cryptographic keys[C]. Proceedings of managing requirements knowledge, International Workshop on, Washington DC: IEEE Computer Society, 1979: 313-313.
- [69] FELDMAN P. A practical scheme for non-interactive verifiable secret sharing[C]. Proceedings of 28th Annual Symposium on Foundations of Computer Science, Piscataway: IEEE, 1987: 427-438.
- [70] ROWSTRON A, DRUSCHEL P. Pastry: Scalable, decentralized object location, and routing for large-scale peer-to-peer systems. [C] Proceedings of IFIP/ACM International Conference on Distributed Systems Platforms and Open Distributed Processing, New York: ACM, 2001: 329-350.
- [71] CAPPOS J, HARTMAN J H. San Fermín: aggregating large data sets using a binomial swap forest [C]. Proceedings of the 5th USENIX Symposium on Networked System Design and Implementation, Berkeley: USENIX, 2008: 147-160.
- [72] BESSANI A, SOUSA J, ALCHIERI E E P. State machine replication for the masses with BFT-SMART. [C] Proceeding of the 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, Piscataway: IEEE, 2014: 355-362.
- [73] 雷元娜, 徐海霞, 李佩丽等区块链共识机制中随机性研究[J]. 信息安全学报, 2021, 6(03): 91-105.
- [74] 刘明瑶, 余益民. 基于区块链与零知识证明的物流隐私保护研究[J]. 软件导刊, 2021, 20(05): 153-157.